Modelos de Calidad de Software

Escuela de Tecnología

Mg. Daniel Arias, **PMP™** y **Scaled Scrum Master™**

¿Qué aprendimos la clase pasada?

- Conceptos
- Arquitectura de Selenium
- Tipos de selenium
- Drivers selenium según explorador
- Localizadores

idat

¿Qué aprendimos la clase pasada?

- Tercera Evaluación Continua

idat

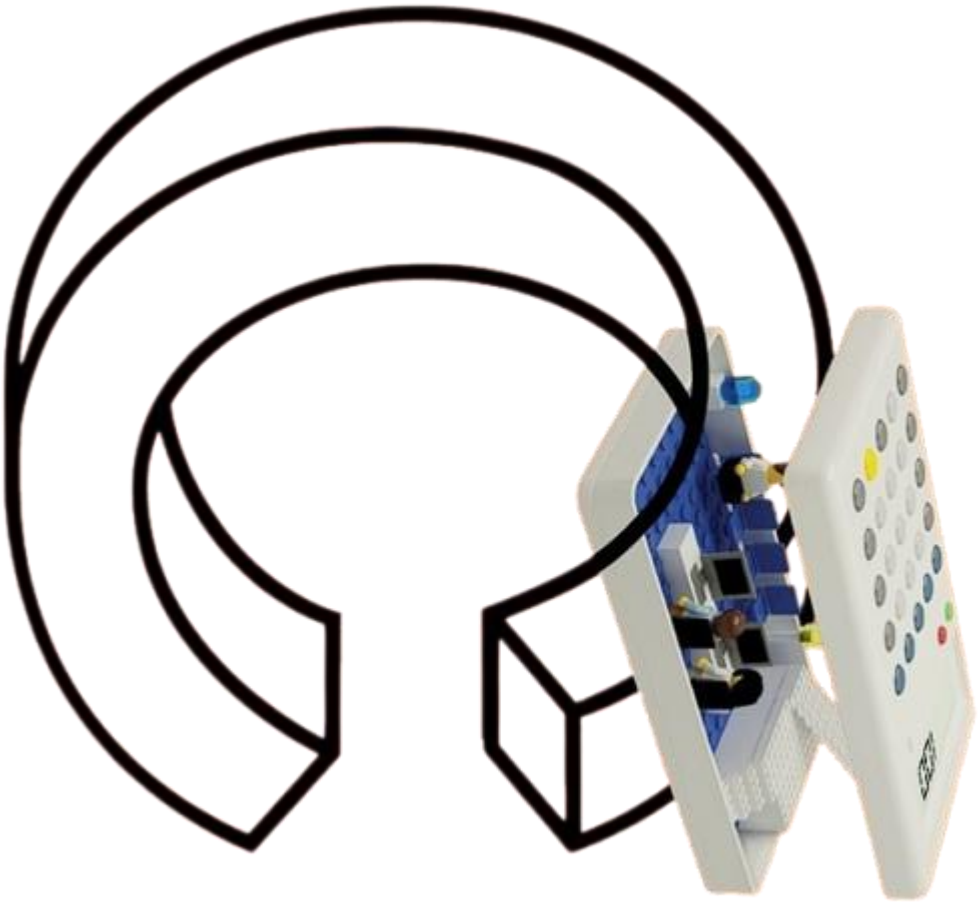
Conversemos

http://www.menti.com

idat

Introducción al Selenium WebDriver – Parte 2

Semana 11



idat

Logro de aprendizaje

- Detectar elementos en la web (Localizadores)
- Creación de scripts de pruebas para certificar la web
- Ejecución del script automatizado
- Integrar JUNIT con Selenium WebDriver
- Revisión de los resultados.
- Características avanzadas de script de pruebas.

Actividad de aprendizaje

ACTIVIDAD VIRTUAL

Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- ¿Qué son las pruebas funcionales?
- Empleando los conocimientos adquiridos en clase, automatizar la web del caso práctico.

idat

Logro de aprendizaje

Interpreta los resultados de la automatización de pruebas realizada, generando los scripts según funcionabilidades de la web con data en duro y dinámica.

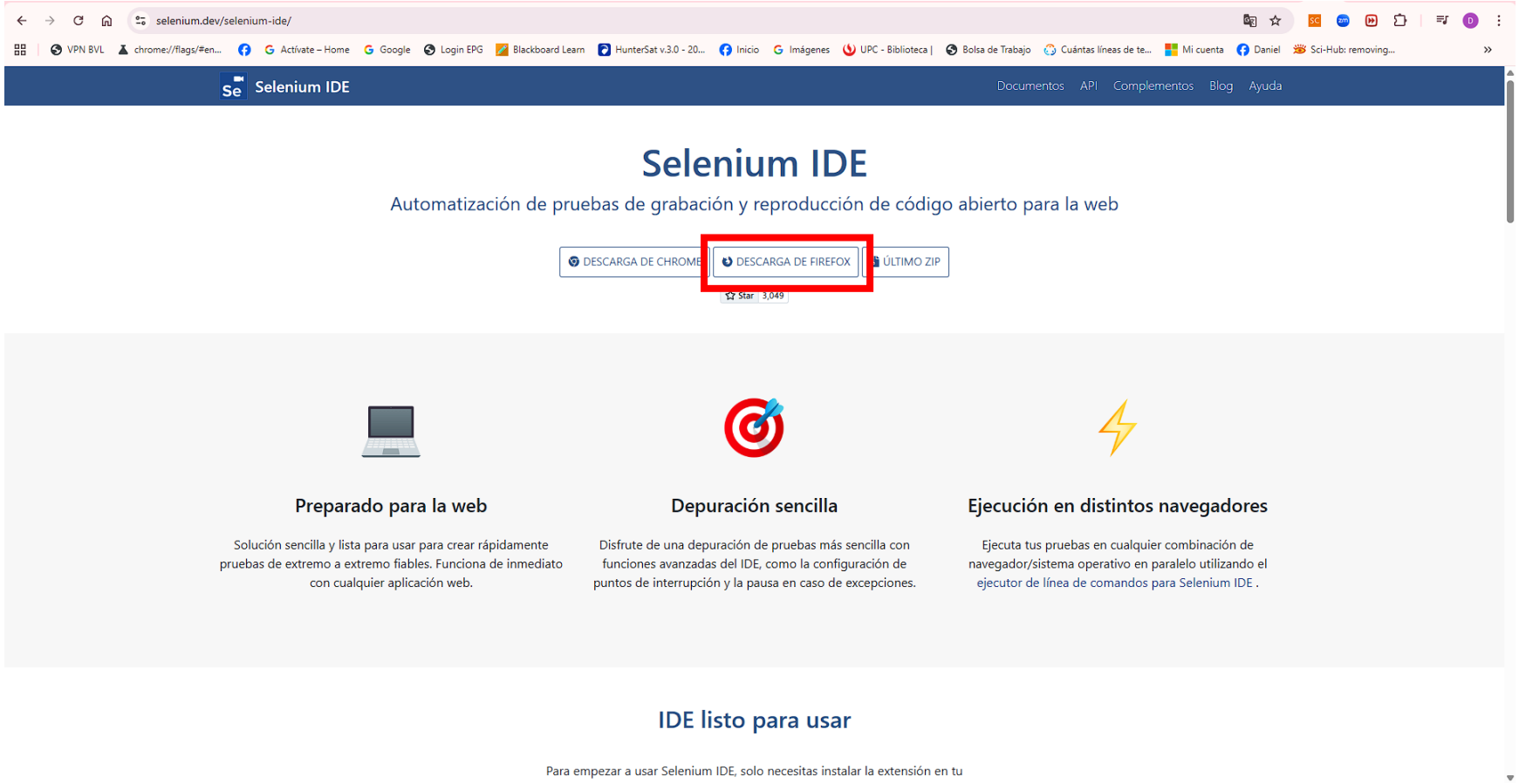
idat

Laboratorio: Instalación de Selenium IDE

idat

Descargamos Selenium IDE

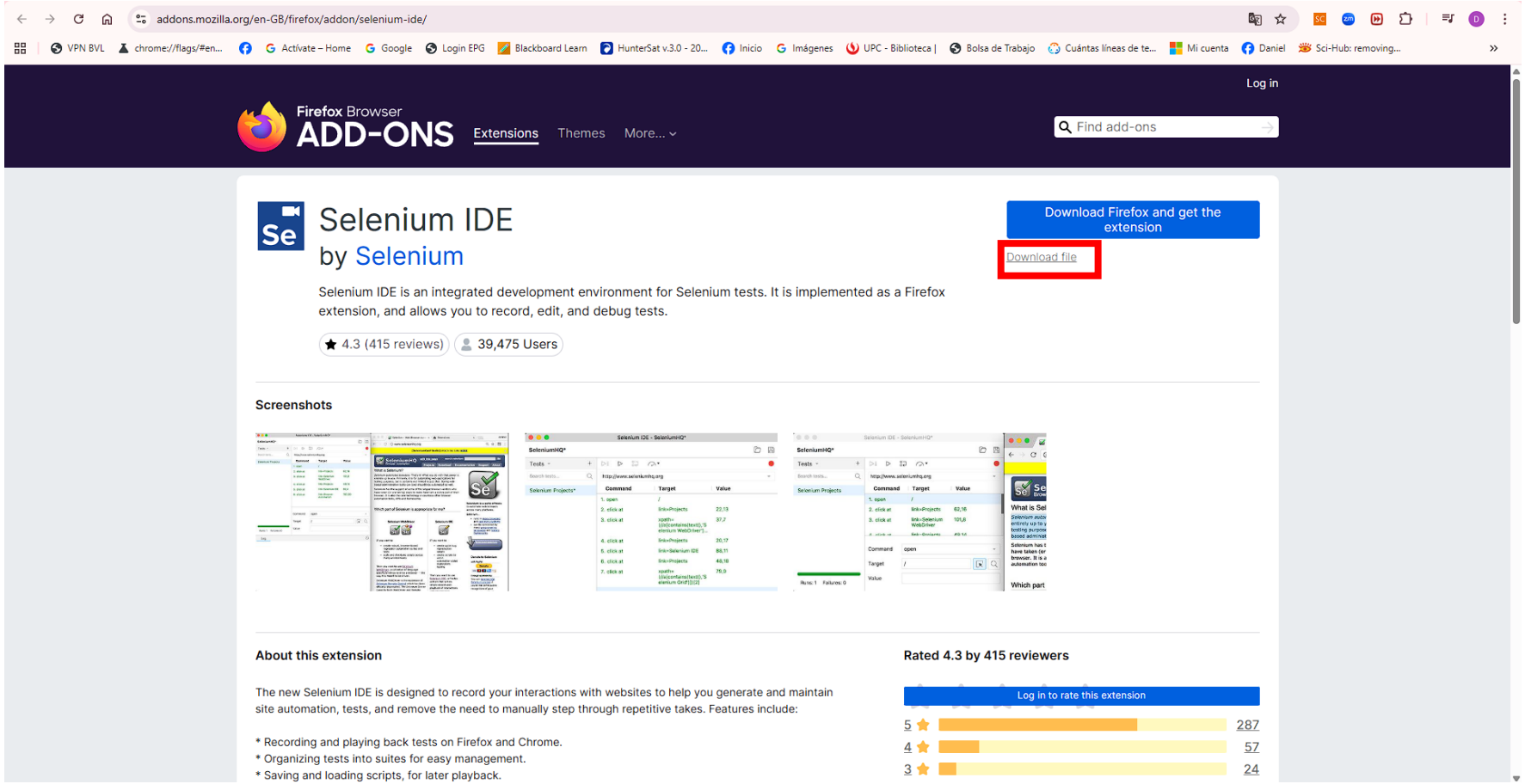
https://www.selenium.dev/selenium-ide/



idat

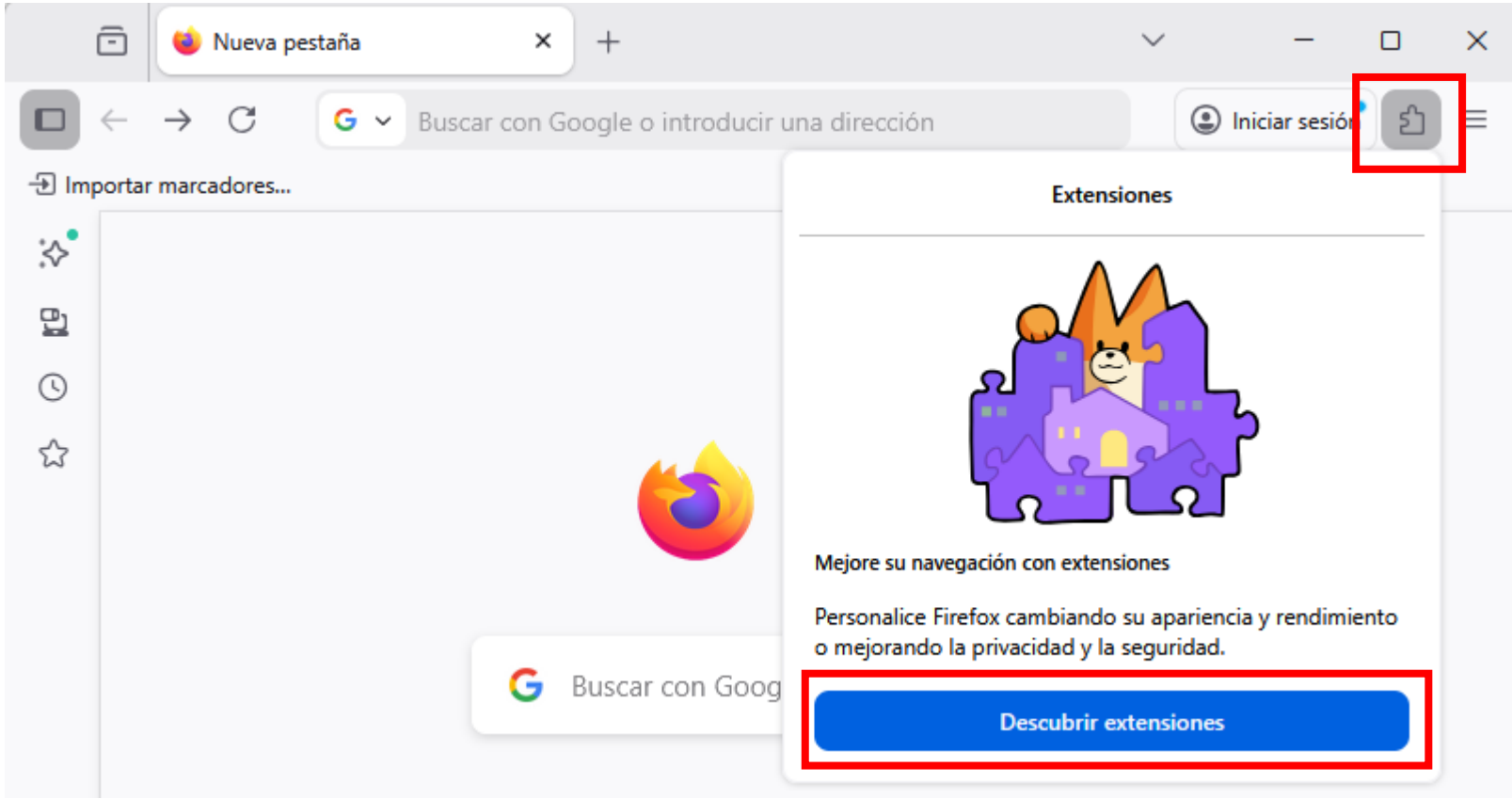
Instalamos la extensión para Firefox

https://addons.mozilla.org/en-GB/firefox/addon/selenium-ide/



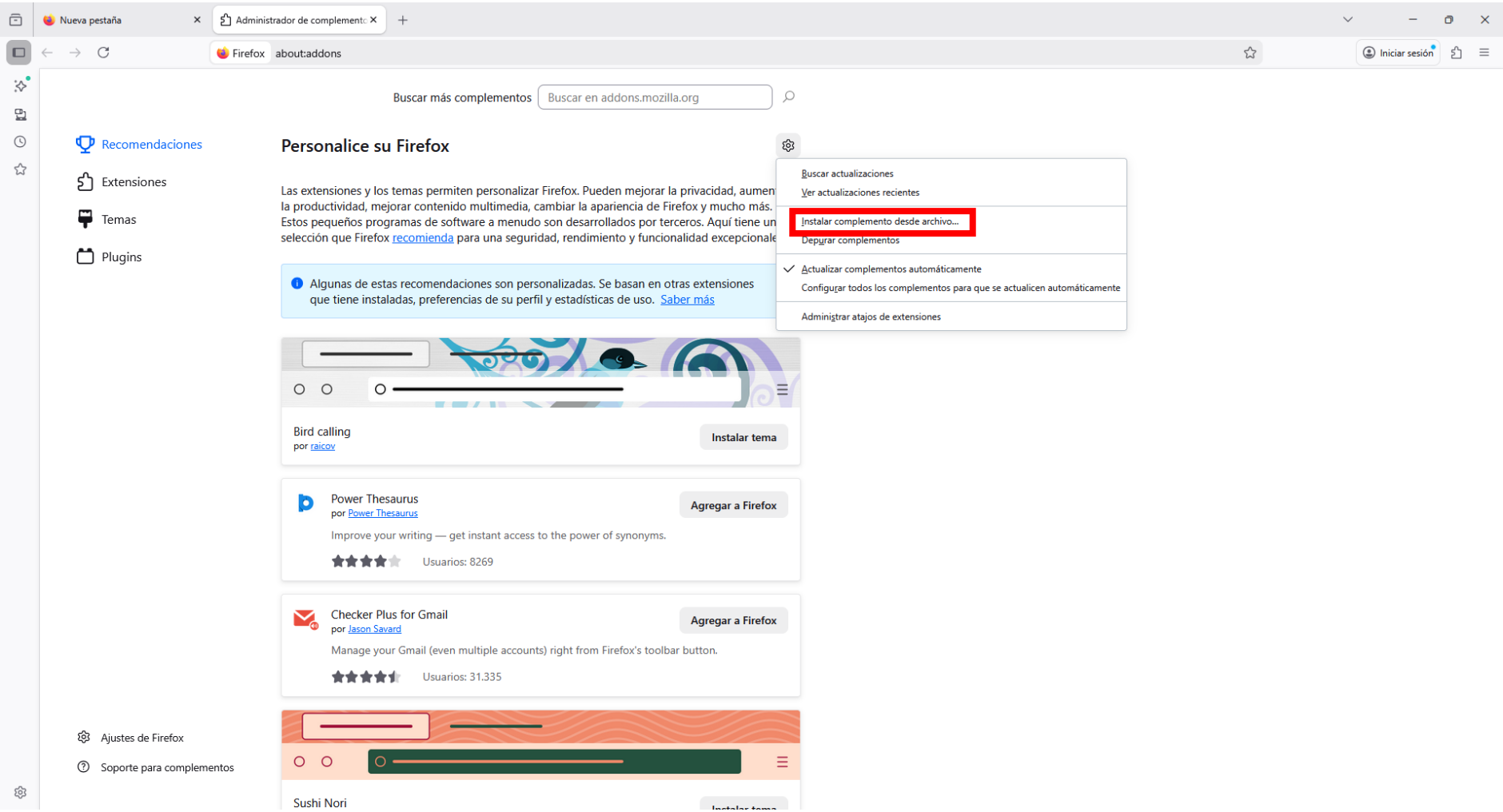
idat

Habilitamos la extensión para Firefox



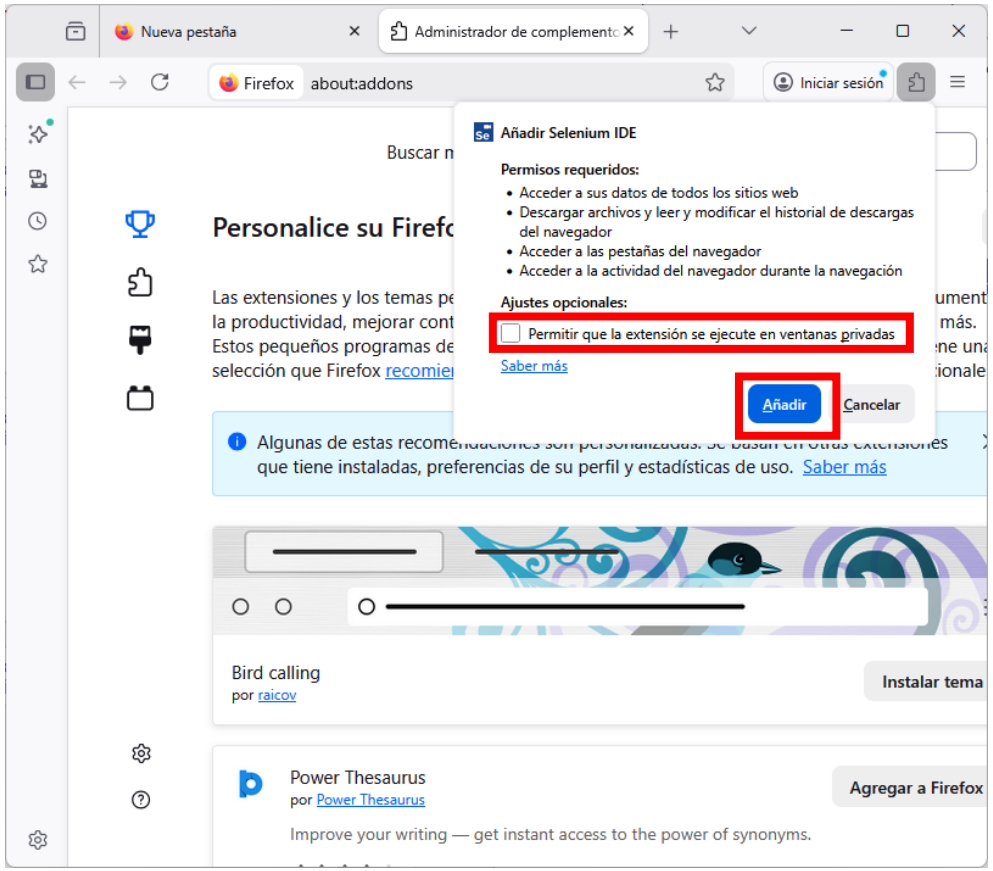
idat

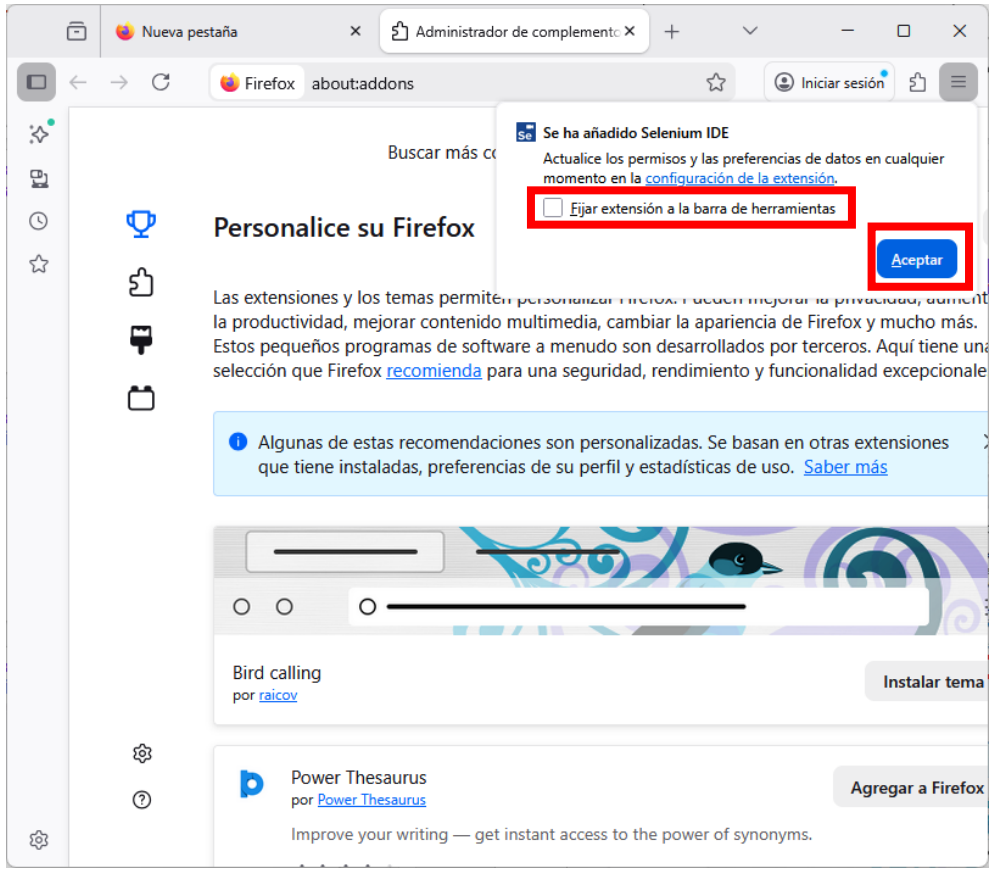
Habilitamos la extensión para Firefox



idat

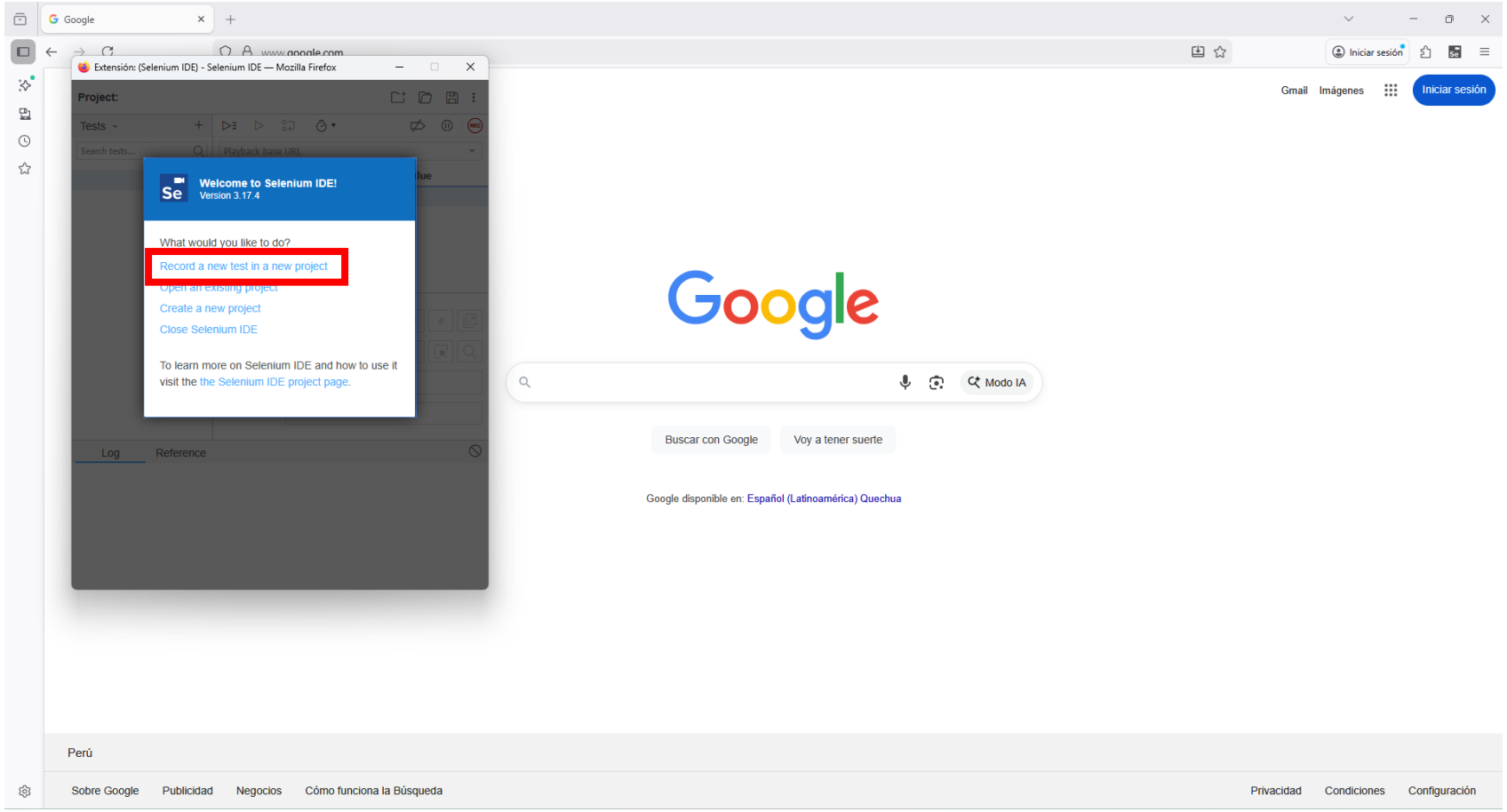
Habilitamos la extensión para Firefox





idat

Bienvenido a Selenium IDE

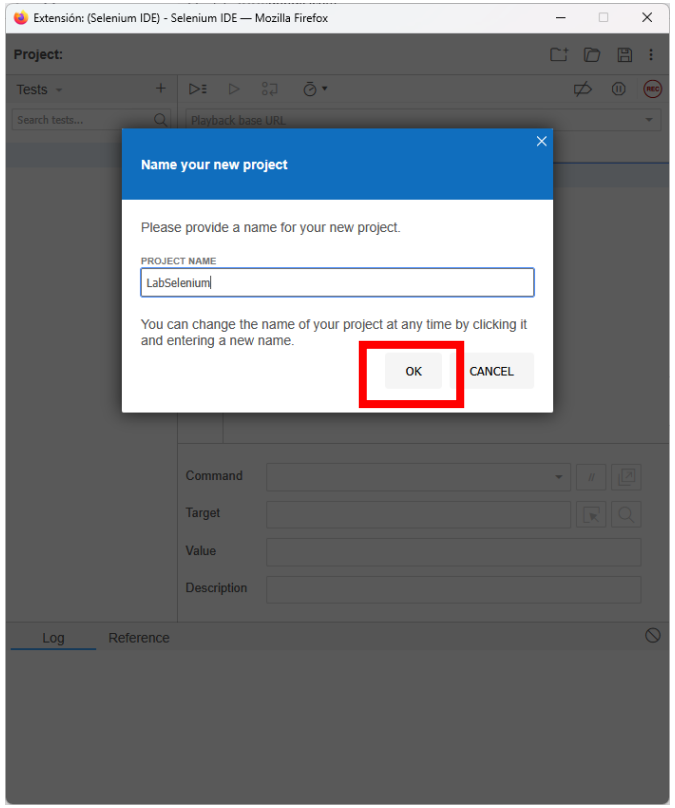


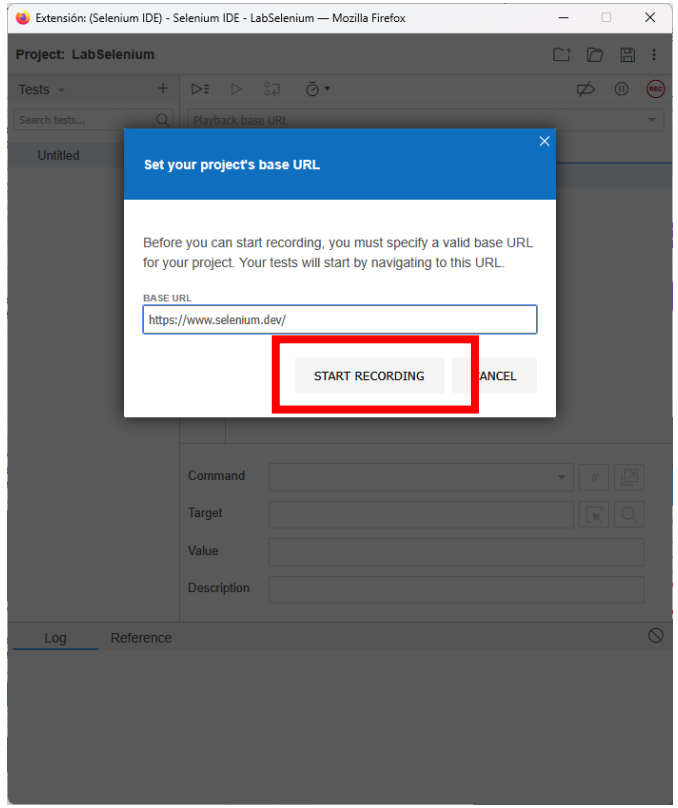
idat

Bienvenido a Selenium IDE

Creamos nuestro primer proyecto

https://www.selenium.dev/

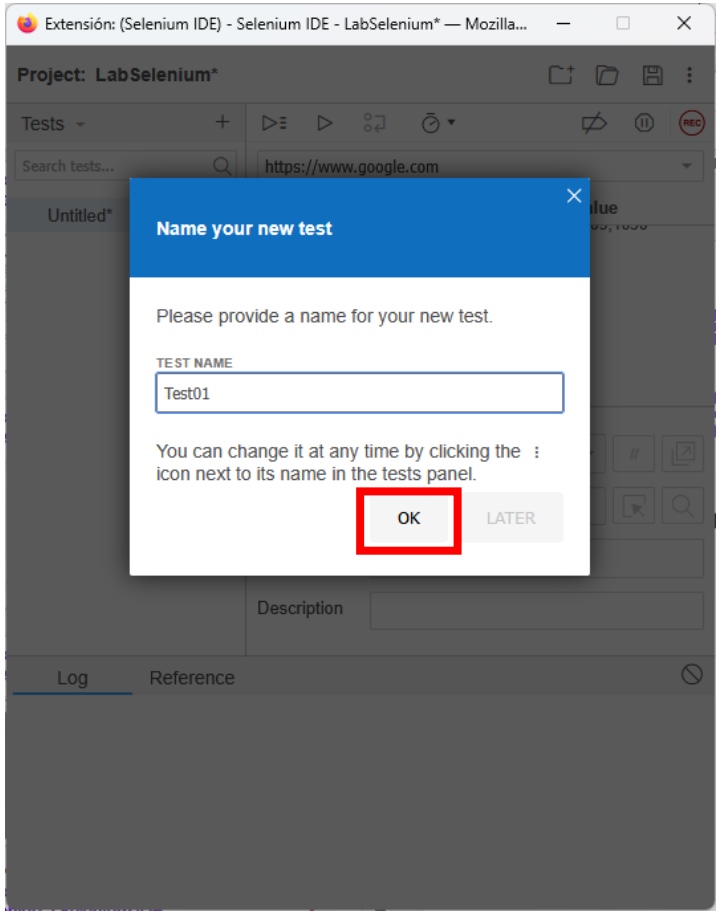


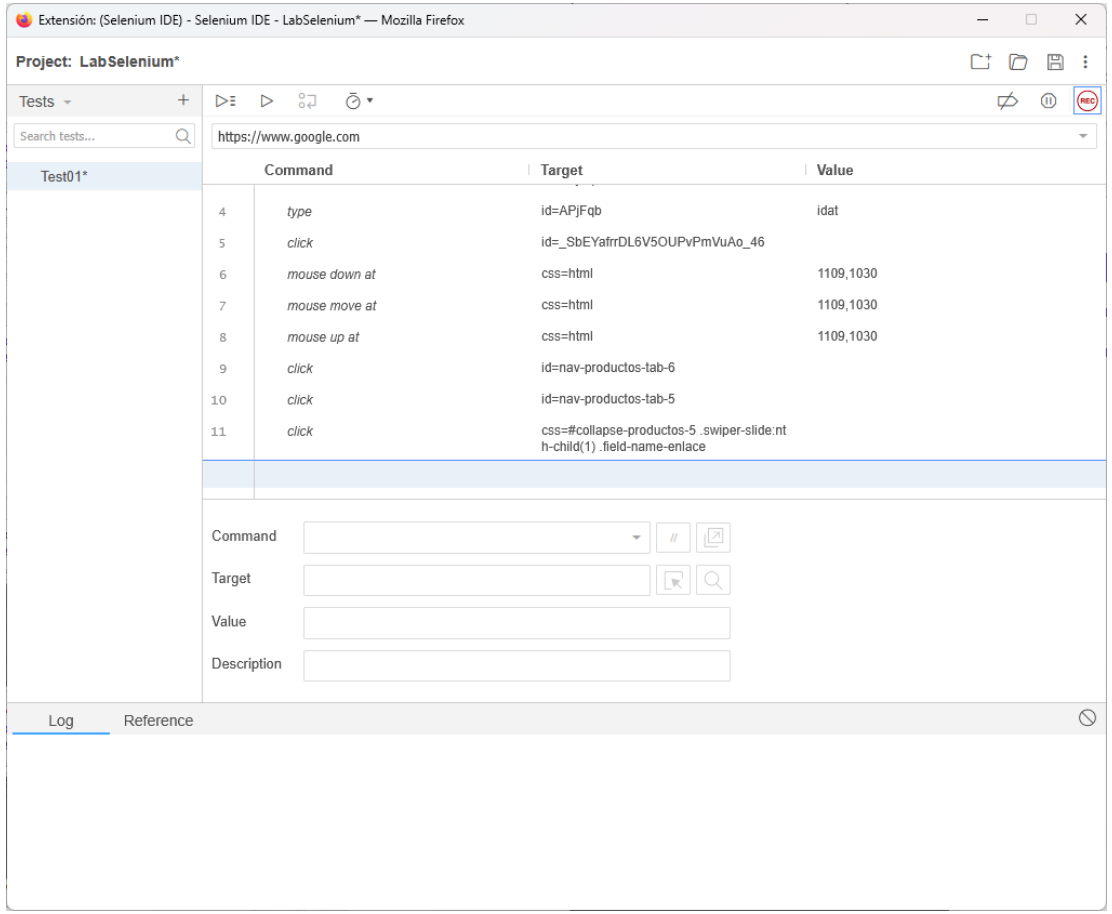


idat

Bienvenido a Selenium IDE

Creamos nuestro primer proyecto

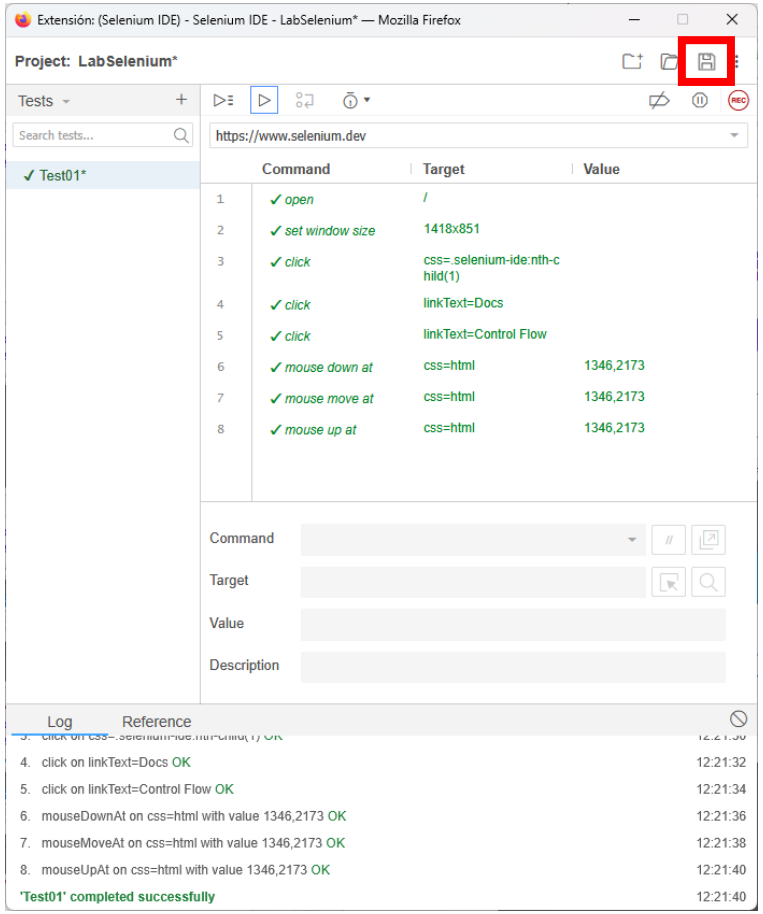


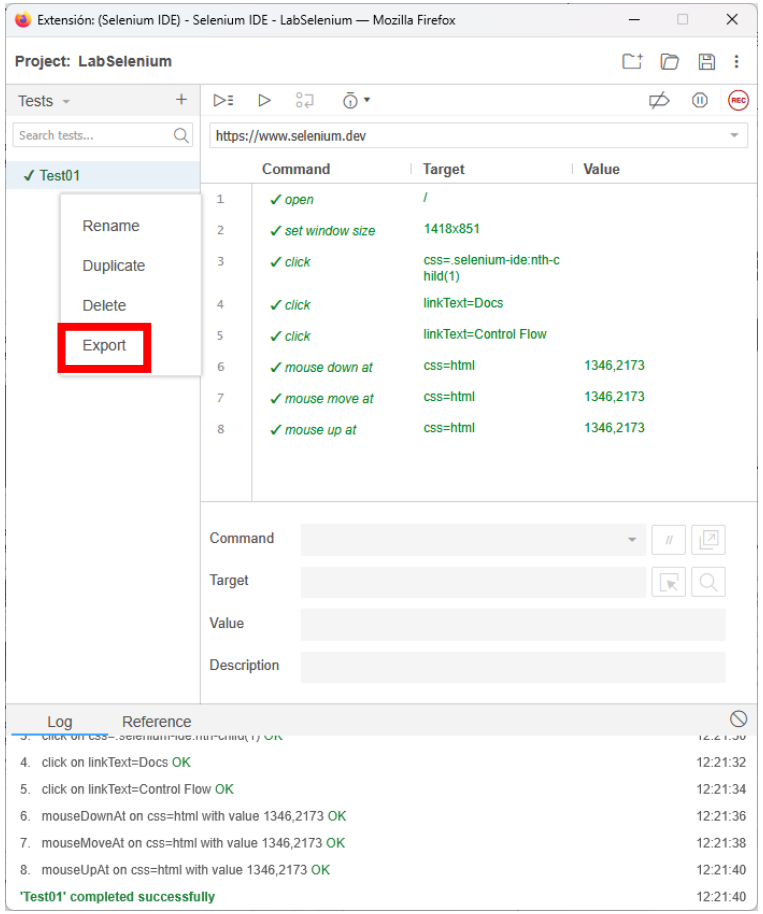


idat

Bienvenido a Selenium IDE

Guardamos nuestro primer proyecto

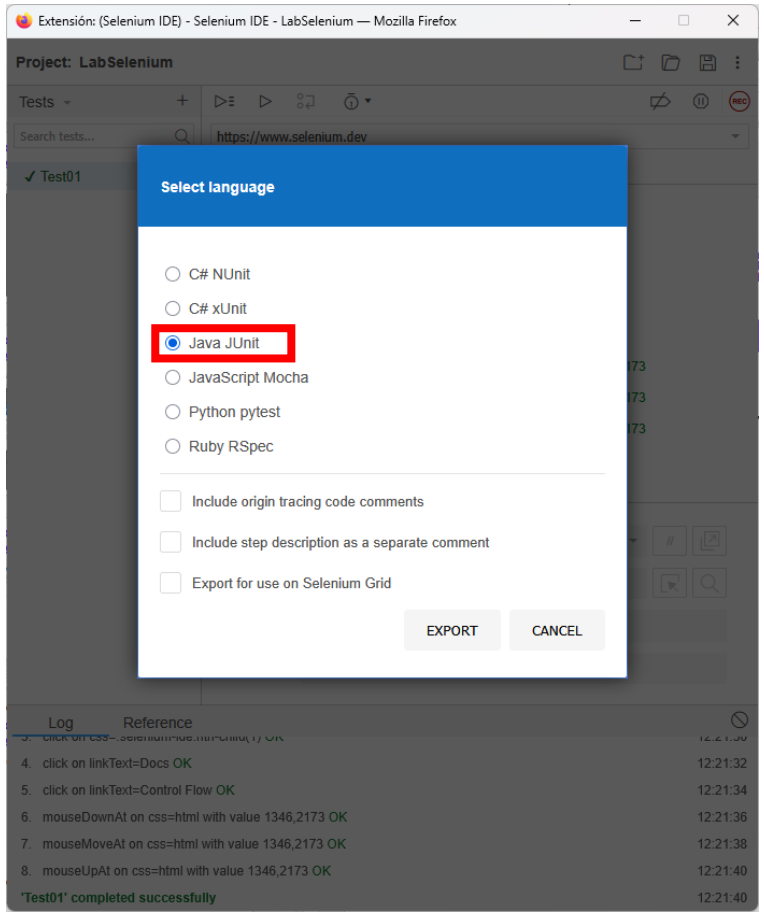




idat

Bienvenido a Selenium IDE

Exportamos nuestro primer proyecto



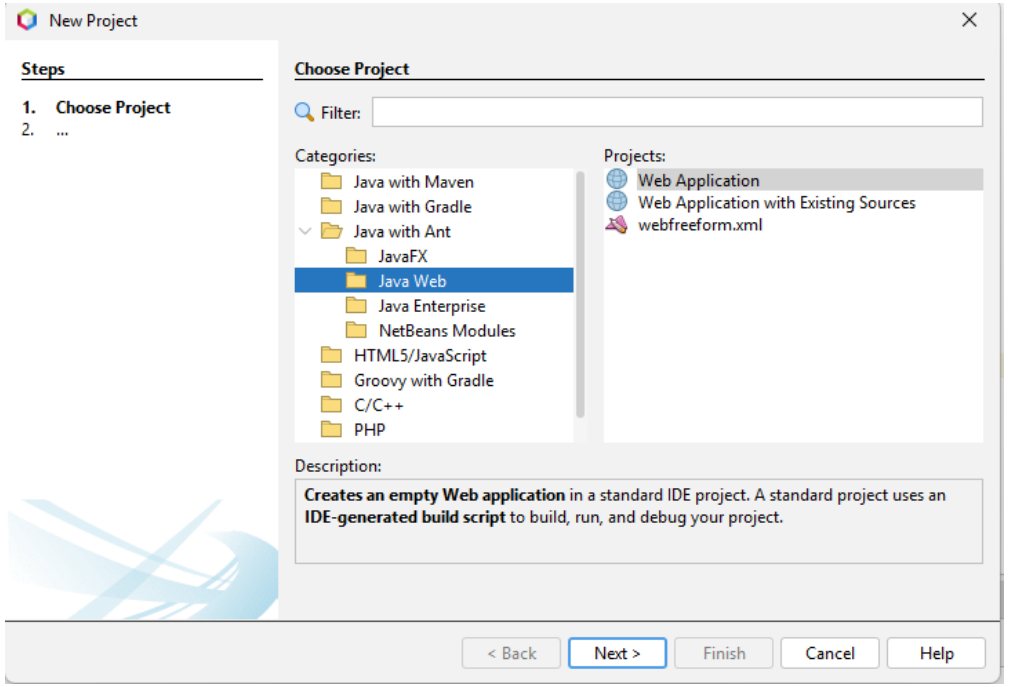
idat

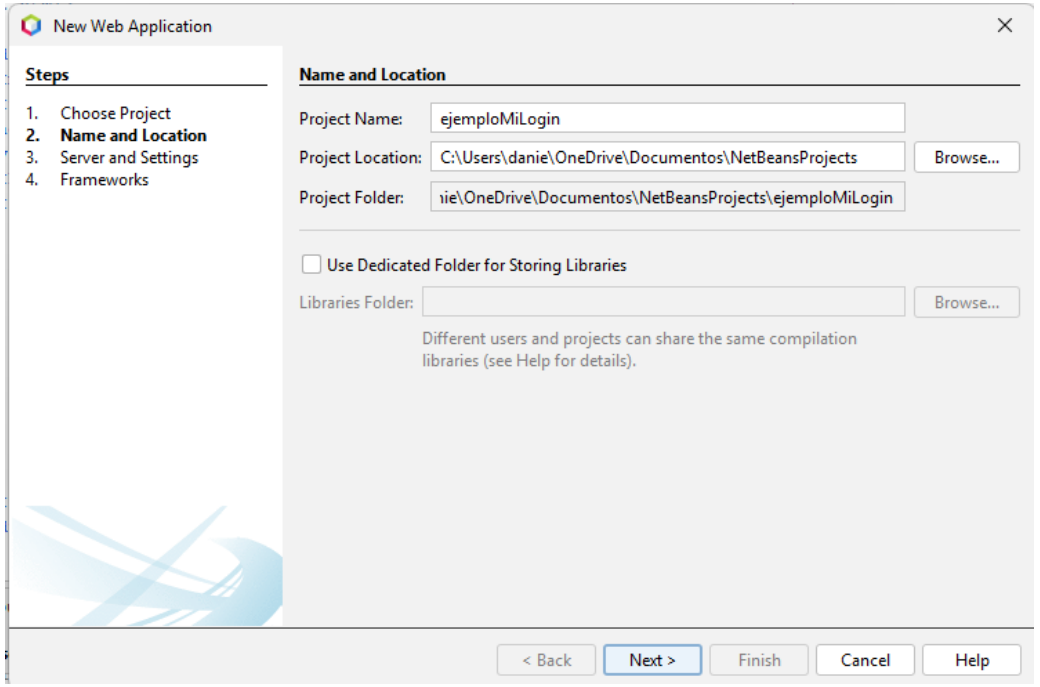
Laboratorio: Selenium y JUnit

idat

Ejecutar Apache Netbeans

- Creamos el Proyecto Java with Ant tipo “Java Web”: “ejemploMiLogin”

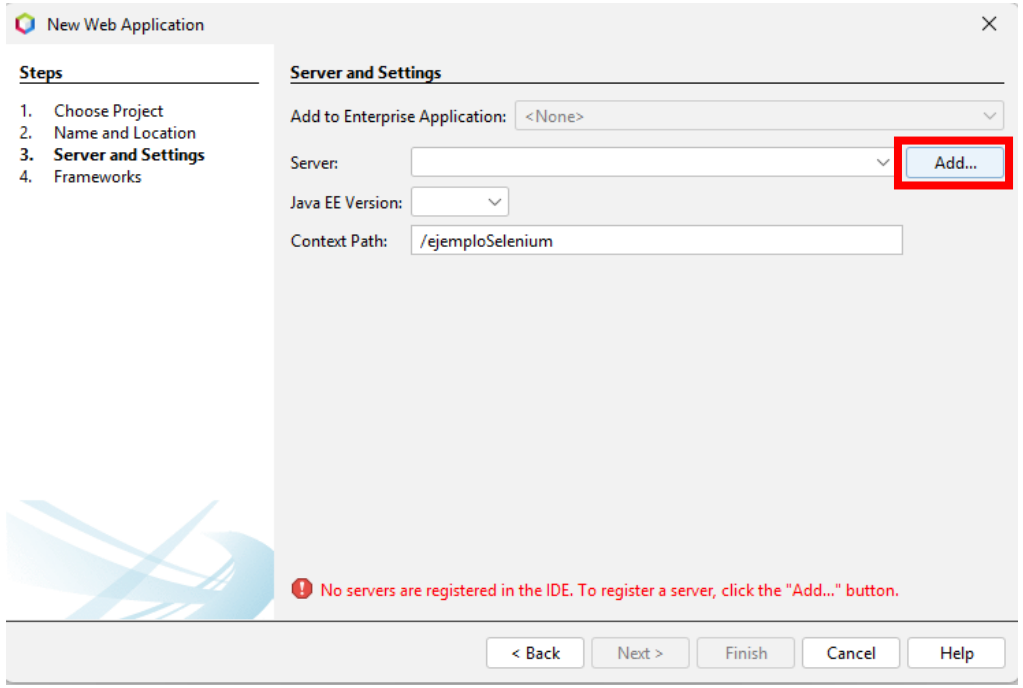


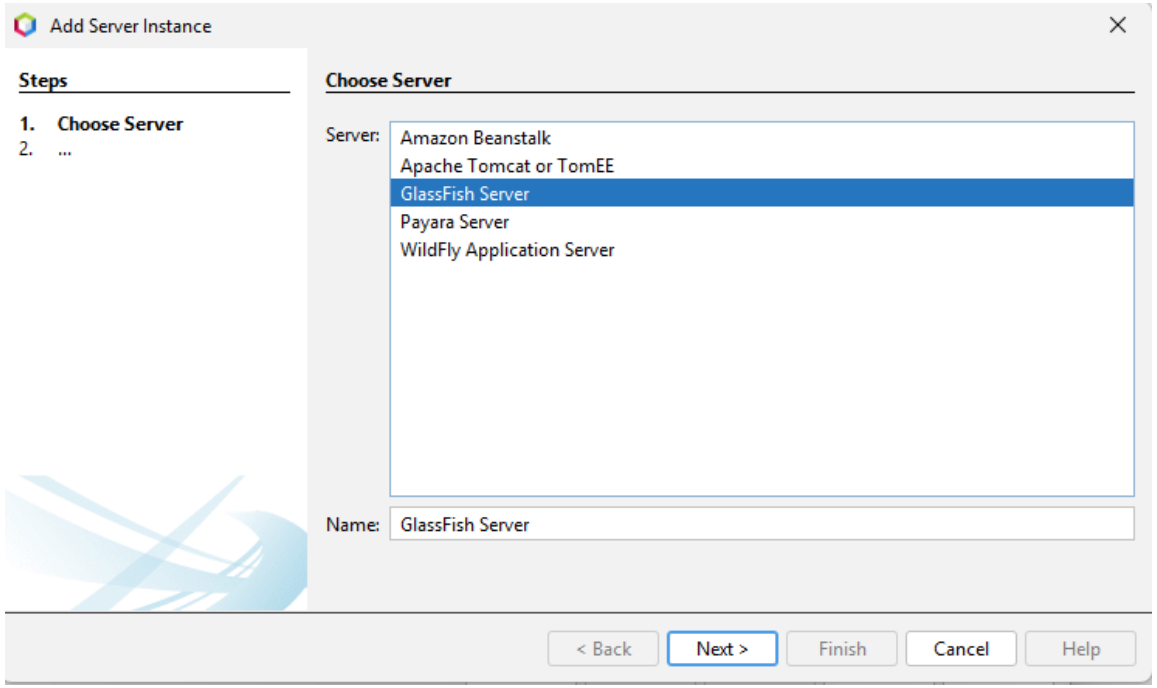


idat

Configuramos el Proyecto

- Si no tenemos configurado un servidor Web, lo configuramos
- Hacemos click en agregar y seleccionamos “GlassFish Server”

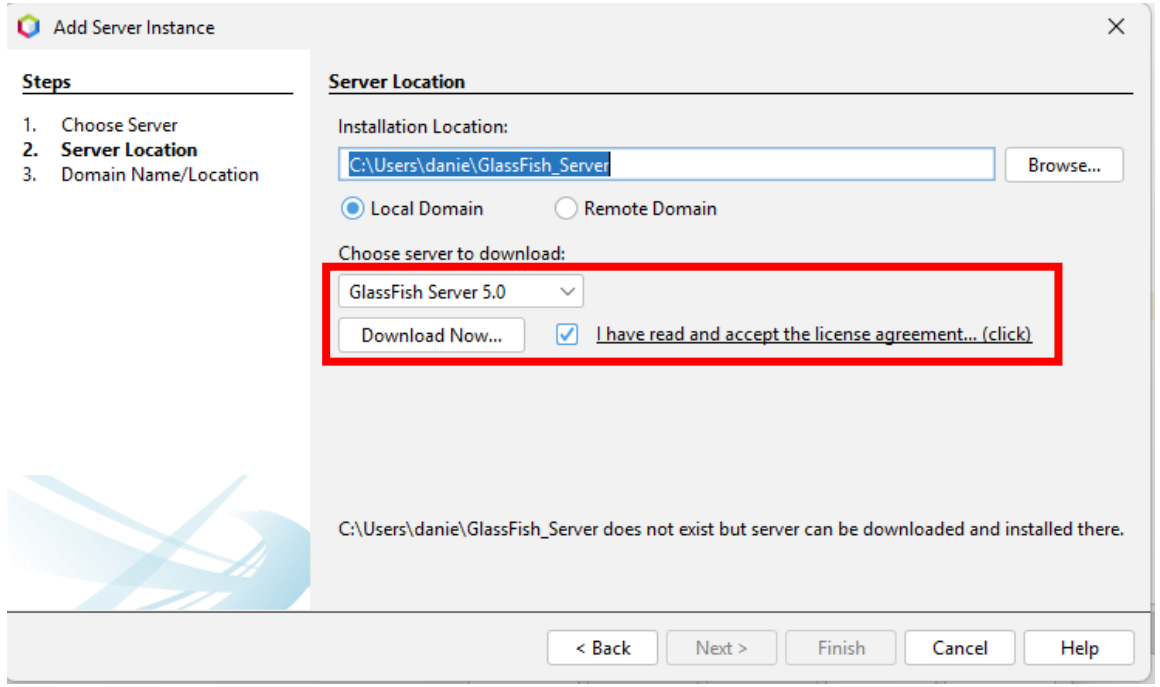


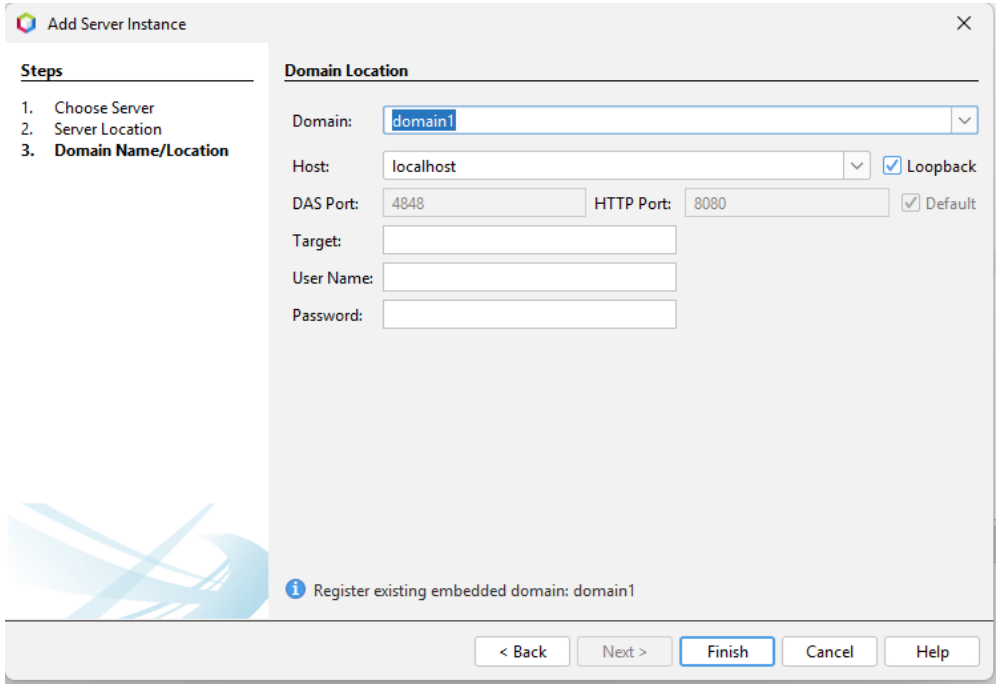


idat

Configuramos el Proyecto

- Descargamos y configuramos el servidor Web

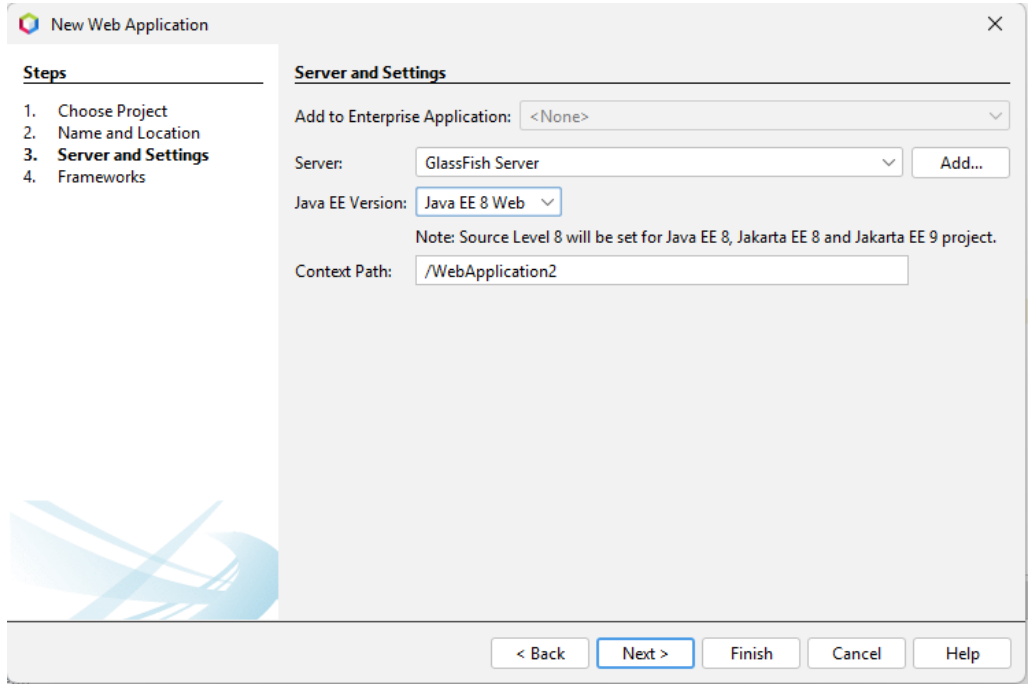


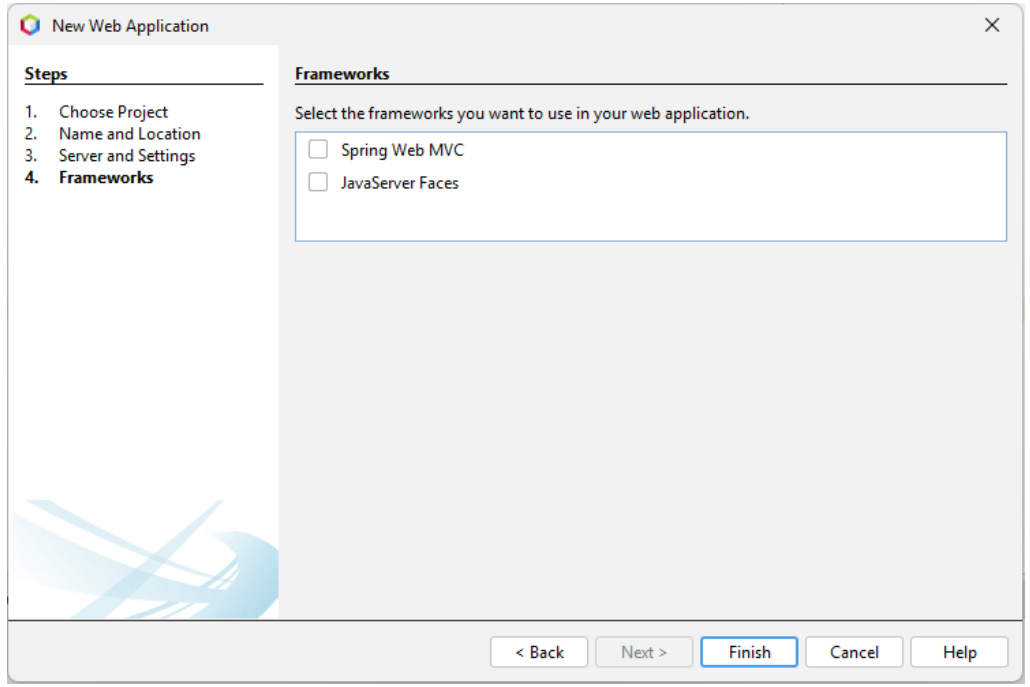


idat

Configuramos el Proyecto

- Seleccionamos el servidor Web “GlassFish Server”
- Y Finalizamos

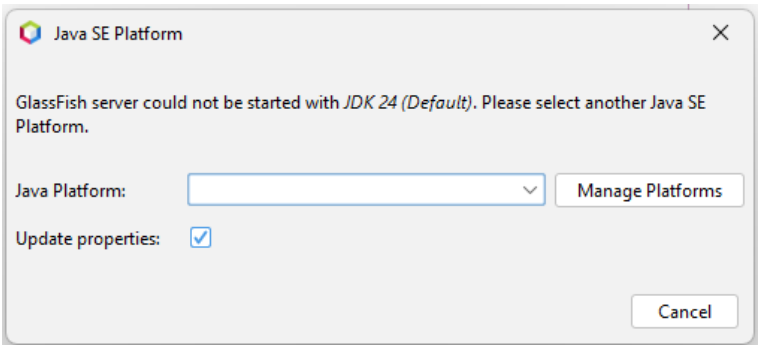


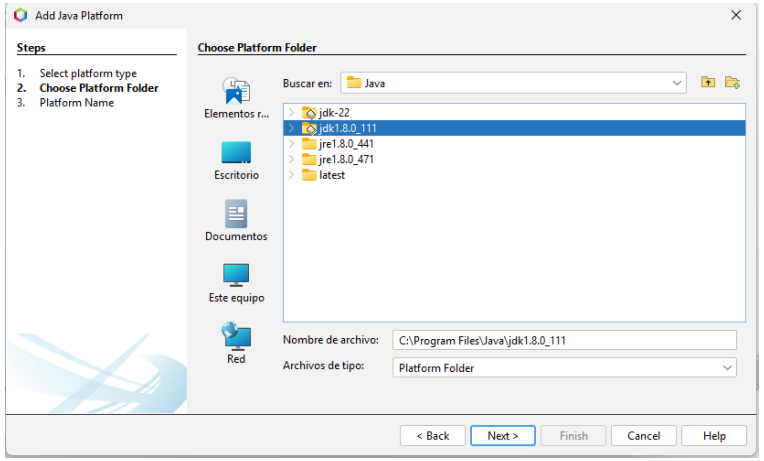


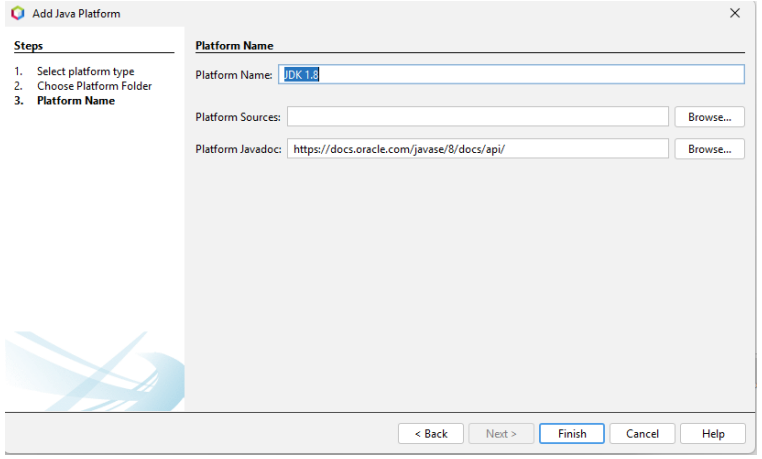
idat

Configuramos el Proyecto

- GlassFish Server 5.0 trabaja con Java 1.8, por lo que –de ser necesario- debemos configurarlo.



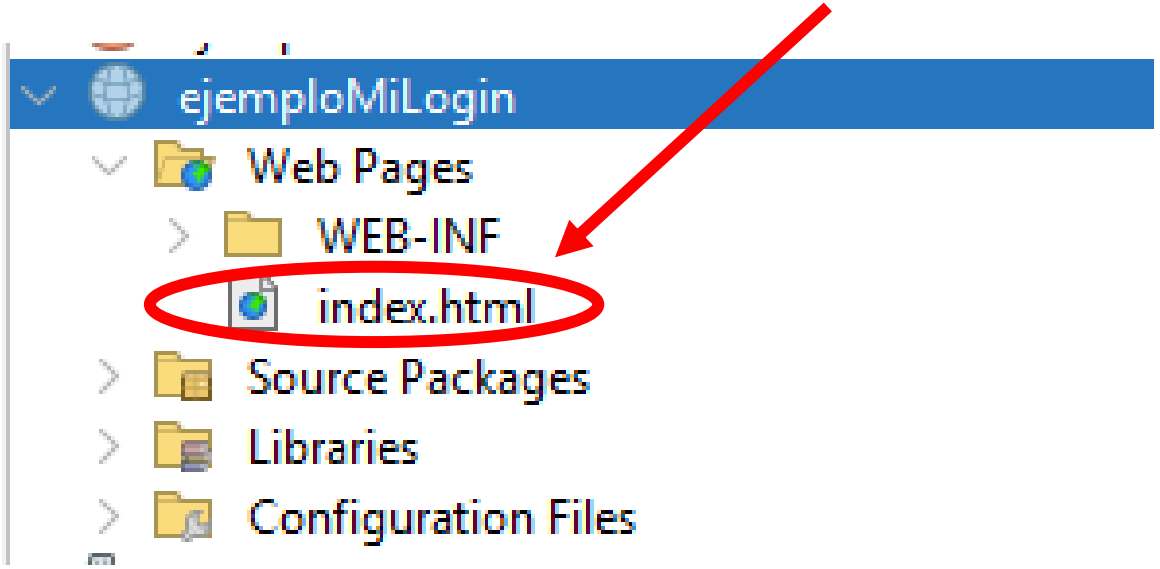




idat

Pasos a seguir

- Como vamos a aplicar un patrón de diseño, eliminamos el archivo index.html

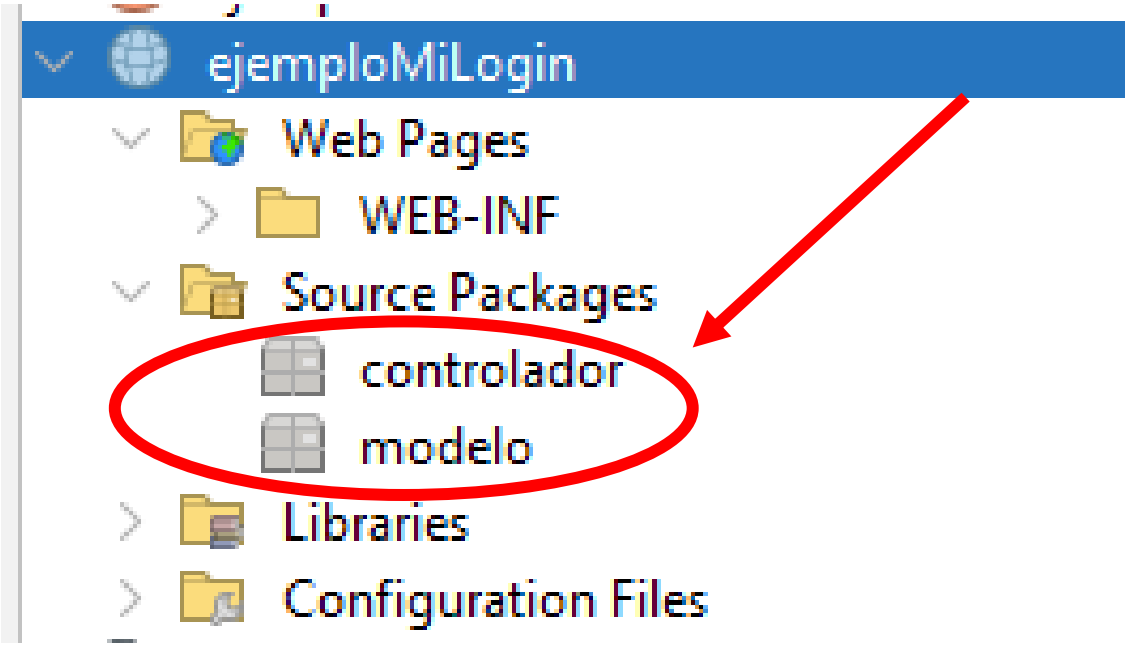


idat

Pasos a seguir

- Creamos las carpetas:
  - controlador
  - modelo

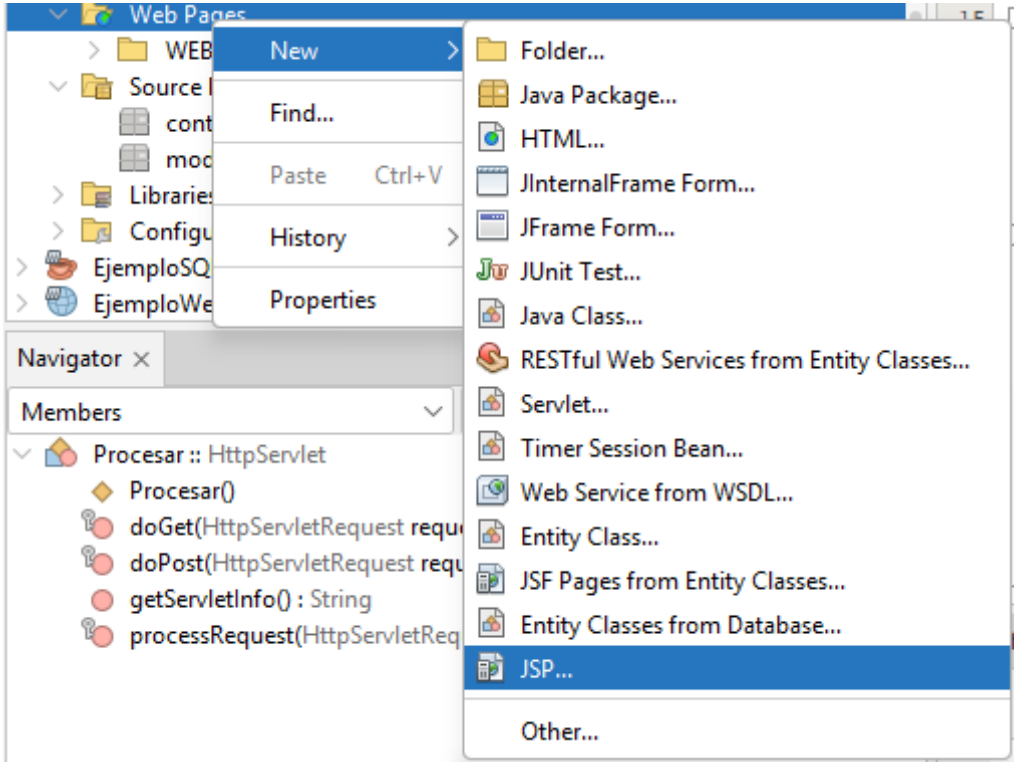
¿Necesitamos crear la carpeta vista?

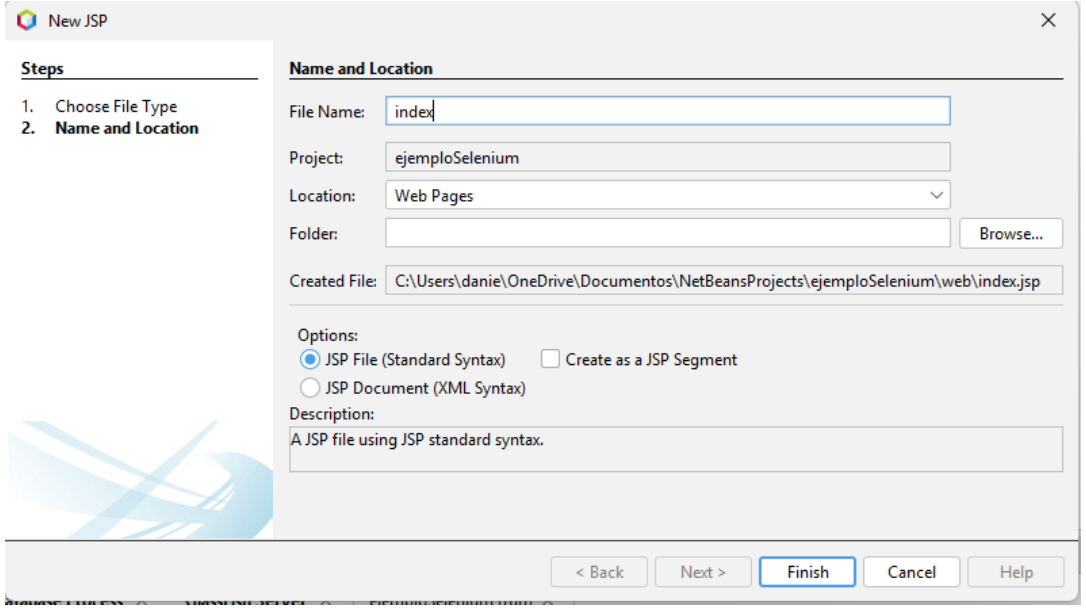


idat

Pasos a seguir

- Creamos nuestra página inicial





idat

Pasos a seguir

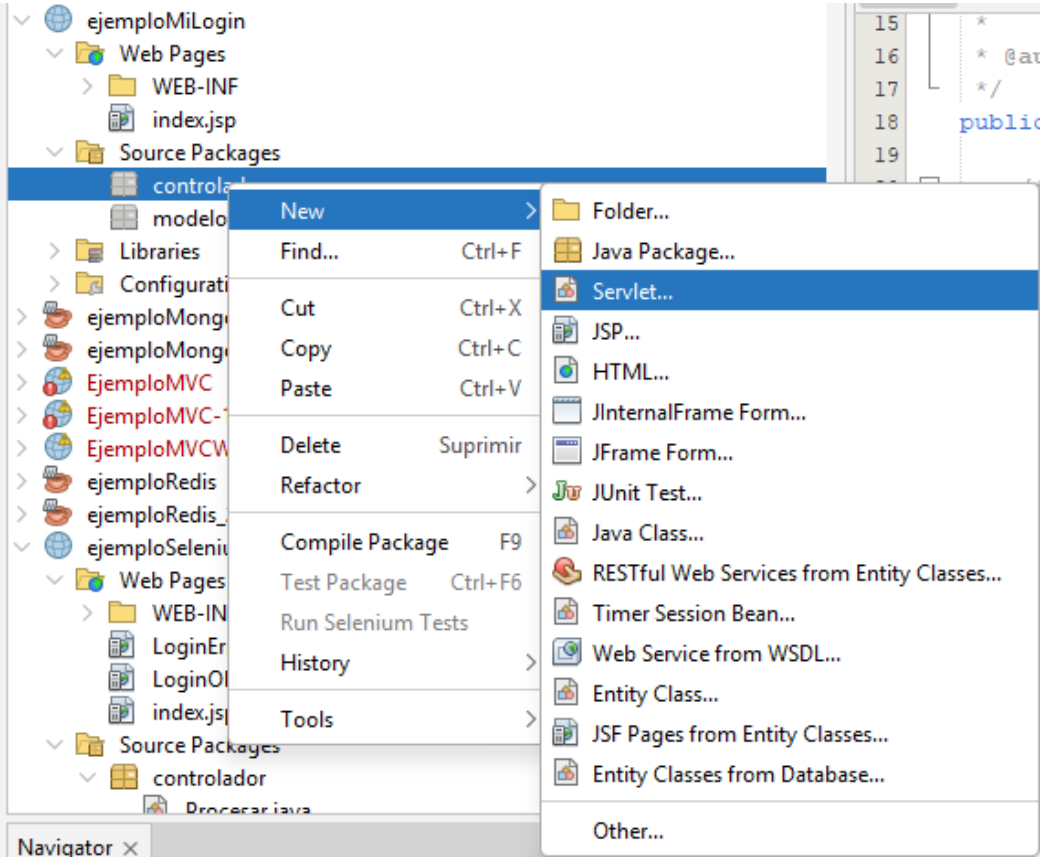
- Codificamos

```
7   <%@page contentType="text/html" pageEncoding="UTF-8"%>
8   <!DOCTYPE html>
9   <html>
10      <head>
11          <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
12          <title>Mi Formulario</title>
13      </head>
14      <body>
15          <h1>Mi Formulario</h1>
16          <form action="procesar.do" method="post">
17              <table>
18                  <tr>
19                      <td>Usuario:</td><td><input type="text" id="txtUsuario" name="txtUsuario" /></td>
20                  </tr>
21                  <tr>
22                      <td>Password</td><td><input type="password" id="txtPassword" name="txtPassword" /></td>
23                  </tr>
24                  <tr>
25                      <td colspan="2"><input type="submit" id="btnEnviar" value="Enviar" /></td>
26                  </tr>
27              </table>
28          </form>
29      </body>
30  </html>
```

idat

Pasos a seguir

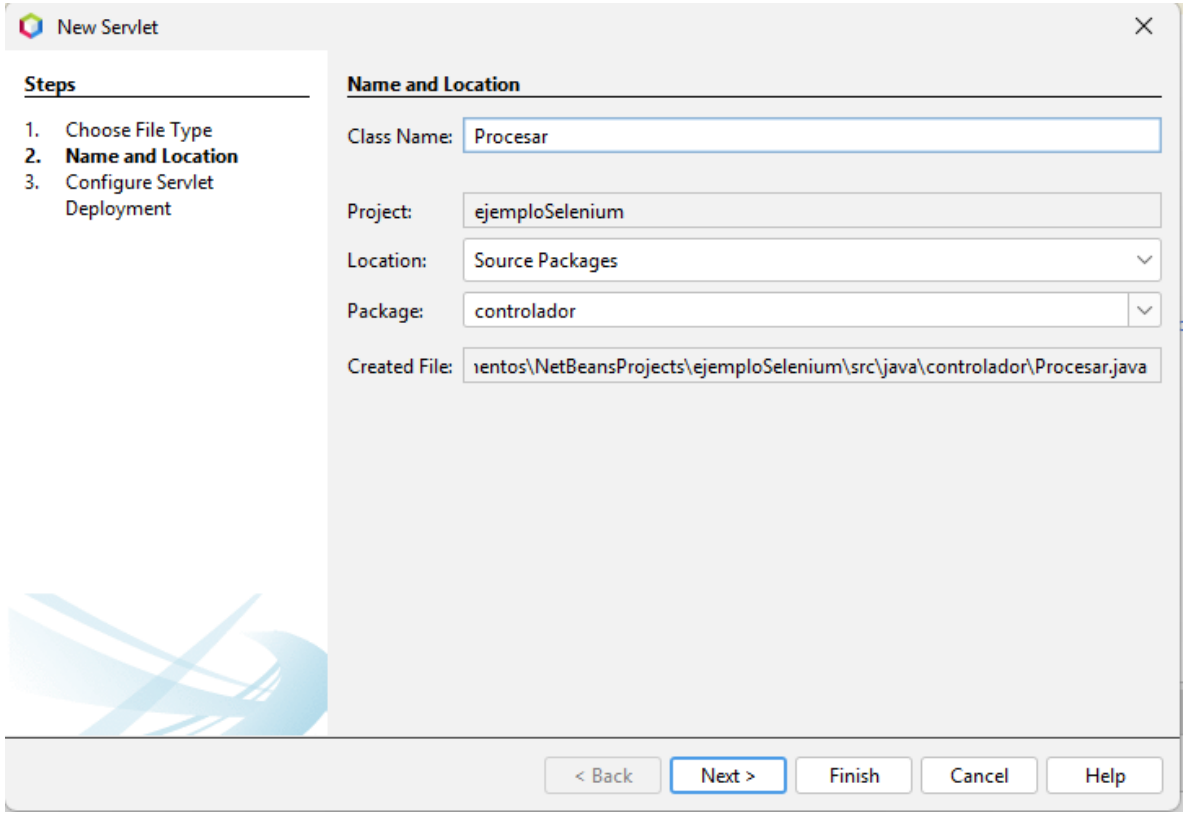
- Creamos ahora el controlador
- Seleccionamos servlet



idat

Pasos a seguir

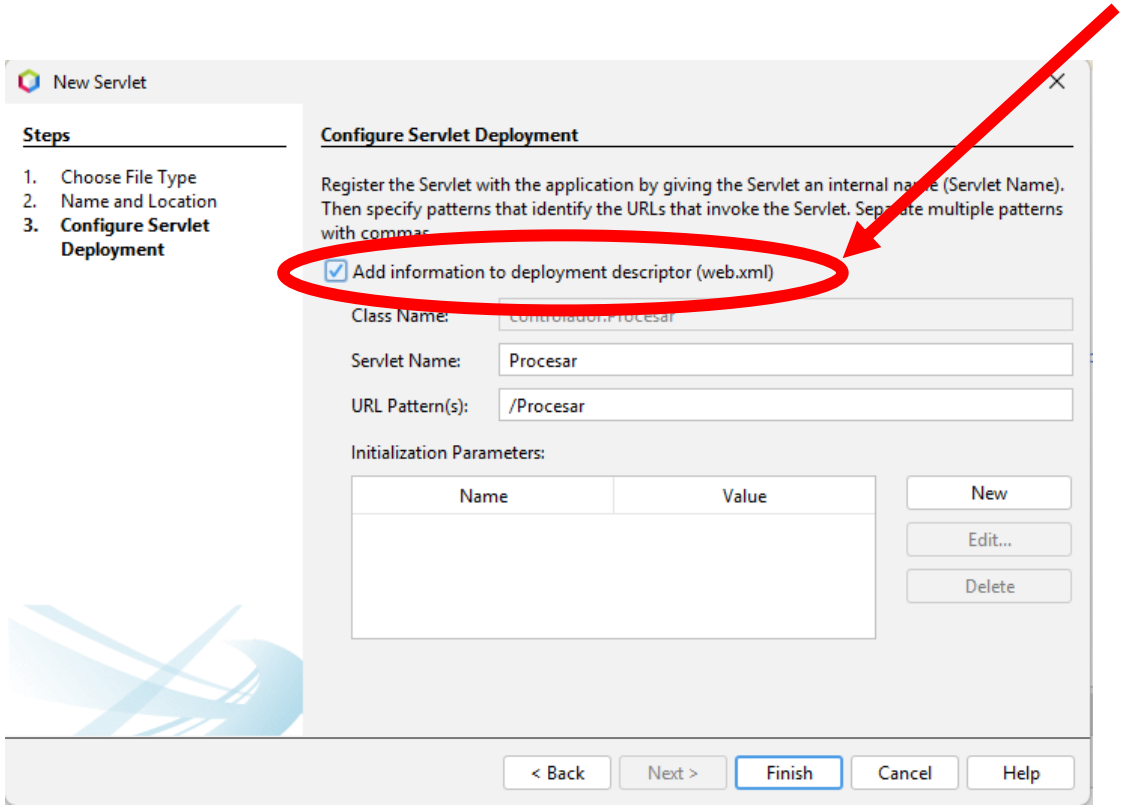
- Colocamos el nombre: Procesar



idat

Pasos a seguir

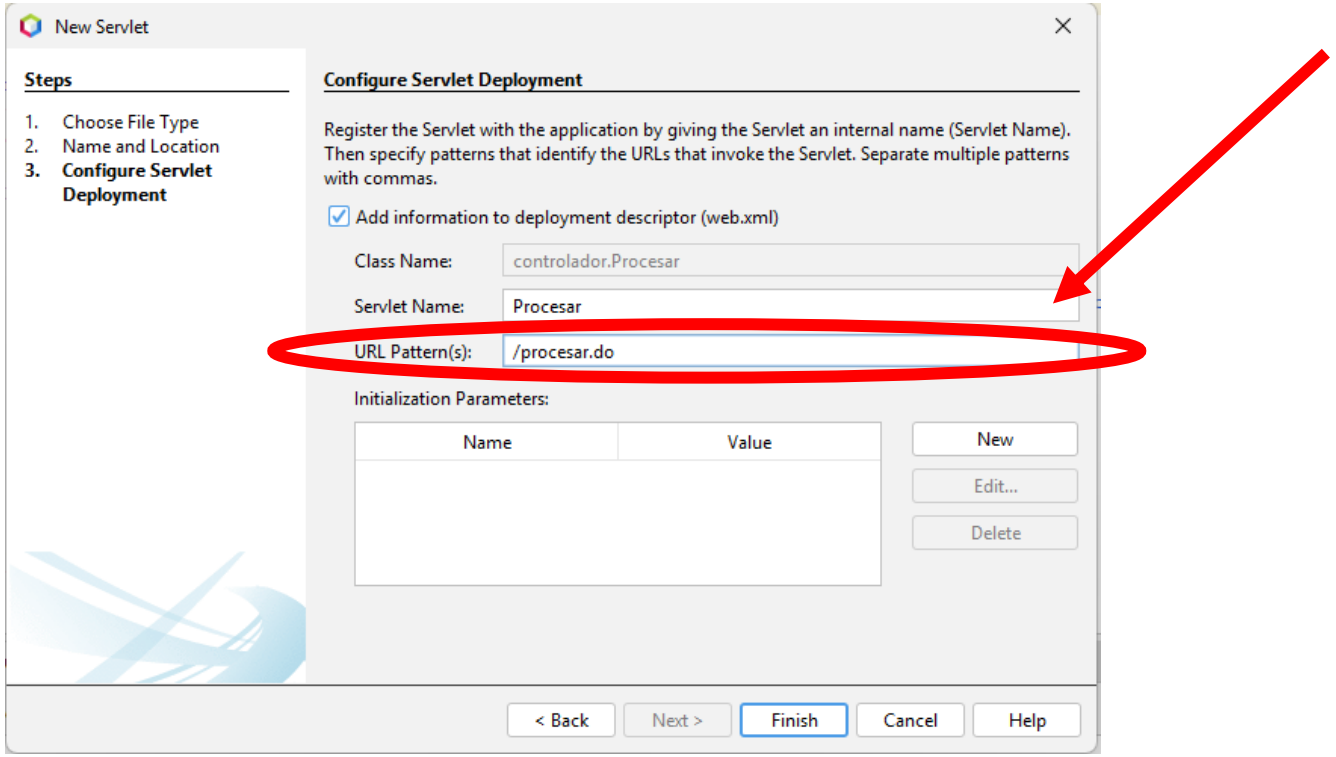
- Ojo, marcar el check de la imagen



idat

Pasos a seguir

- Cambiamos el nombre en el URL Pattern y extensión .do



idat

Pasos a seguir

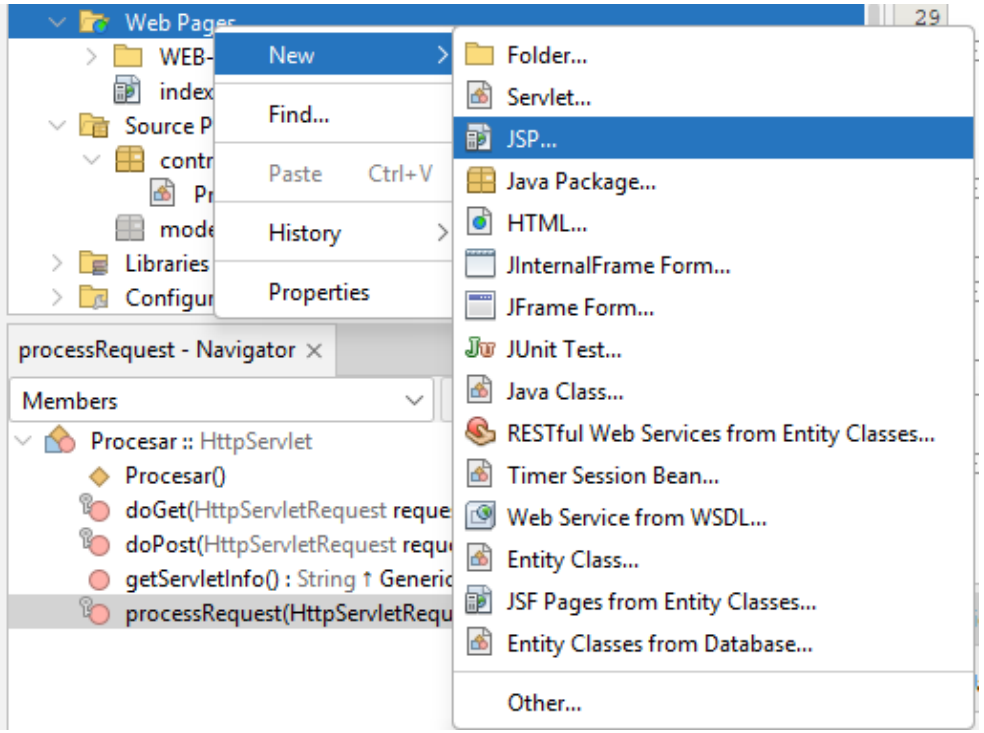
- Eliminamos el código que nos ha creado el IDE
- ¿Por qué?

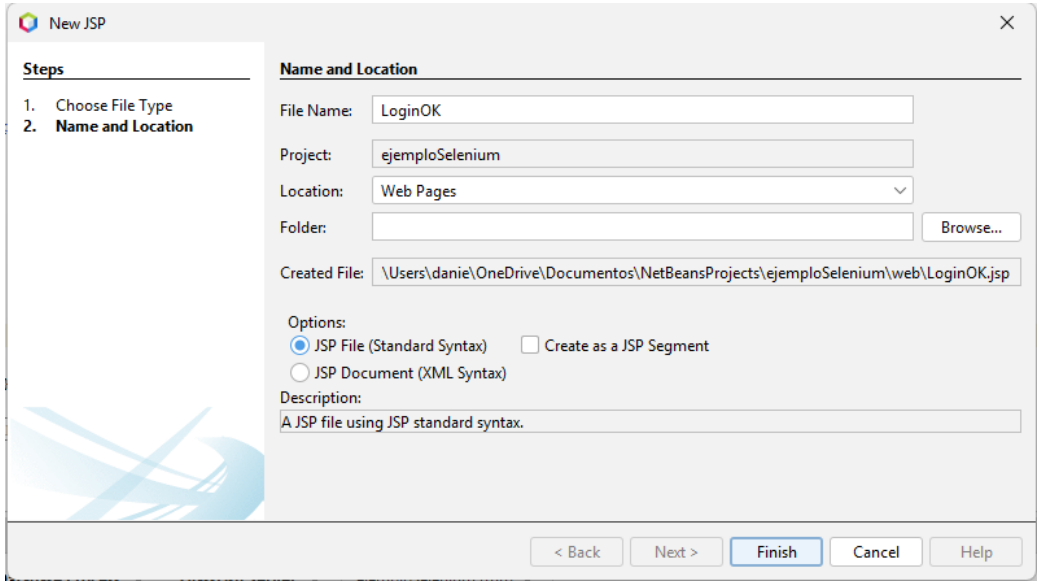
```
29      protected void processRequest(HttpServletRequest request, HttpServletResponse response)
30              throws ServletException, IOException {
31          response.setContentType("text/html;charset=UTF-8");
32          try (PrintWriter out = response.getWriter()) {
33              /* TODO output your page here. You may use following sample code. */
34              out.println("<!DOCTYPE html>");
35              out.println("<html>");
36              out.println("<head>");
37              out.println("<title>Servlet Procesar</title>");
38              out.println("</head>");
39              out.println("<body>");
40              out.println("<h1>Servlet Procesar at " + request.getContextPath() + "</h1>");
41              out.println("</body>");
42              out.println("</html>");
43          }
44      }
```

idat

Pasos a seguir

- Regresamos a la Capa Vista
- Crea el JSP “LoginOK”





idat

Pasos a seguir

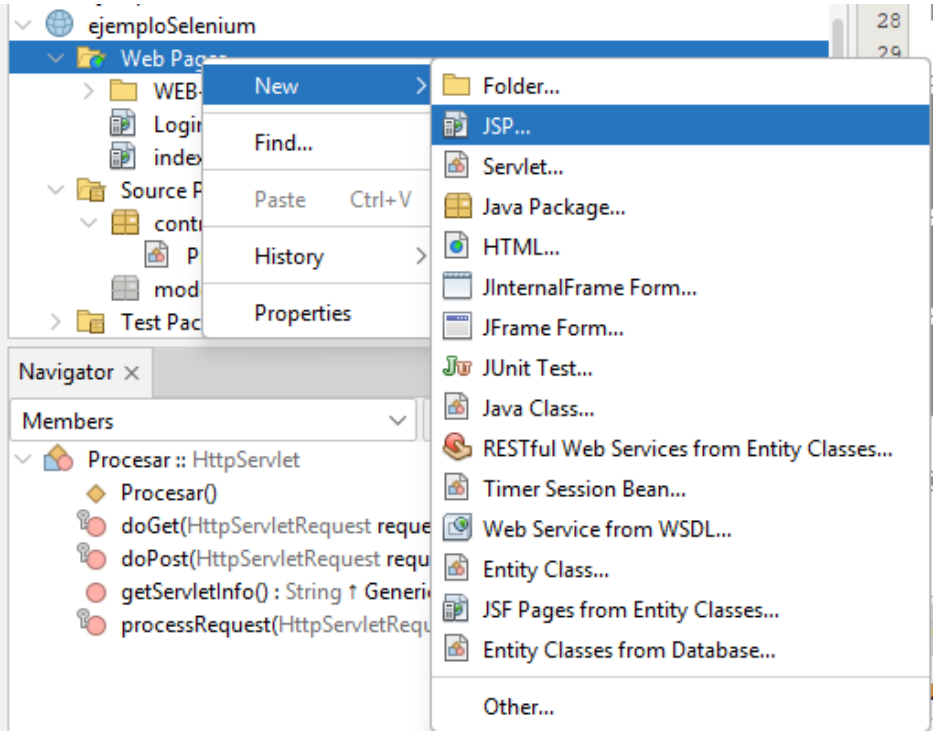
- Codificamos

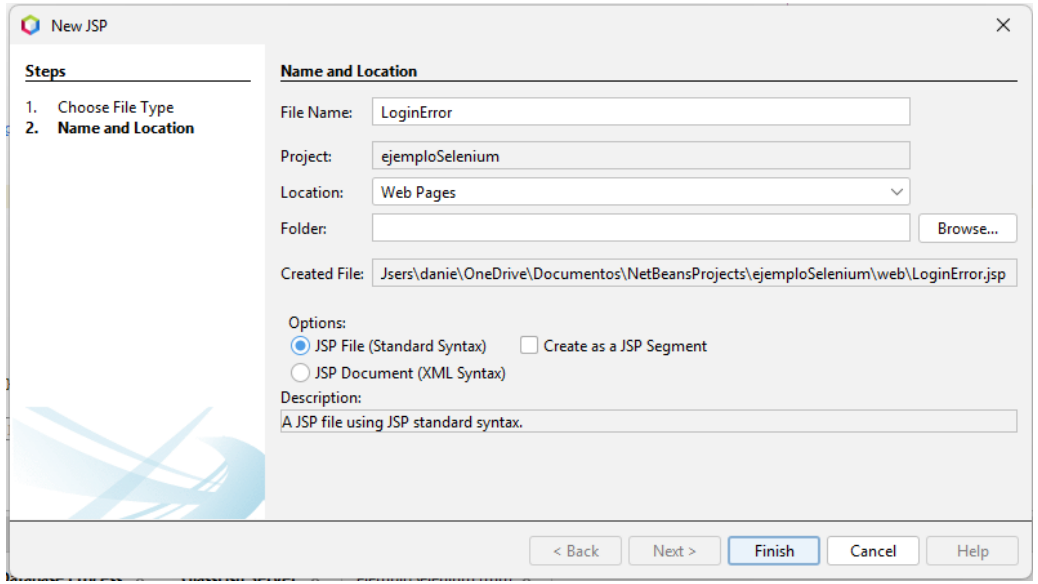
```
7   <%@page contentType="text/html" pageEncoding="UTF-8"%>
8   <!DOCTYPE html>
9   <html>
10      <head>
11          <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
12          <title>Login OK</title>
13      </head>
14      <body>
15          <h1>Usuario y Contraseña correcta</h1>
16          <br>
17          <a href="index.jsp">Volver...</a>
18      </body>
19  </html>
```

idat

Pasos a seguir

- Nuevamente en la Capa Vista
- Crea el JSP “LoginError”





idat

Pasos a seguir

- Codificamos

```
7   <%@page contentType="text/html" pageEncoding="UTF-8"%>
8   <!DOCTYPE html>
9   <html>
10      <head>
11          <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
12          <title>Login Incorrecto</title>
13      </head>
14      <body>
15          <h1>Usuario y Contraseña inválidos</h1>
16          <br>
17          <a href="index.jsp">Volver...</a>
18      </body>
19  </html>
```

idat

Pasos a seguir

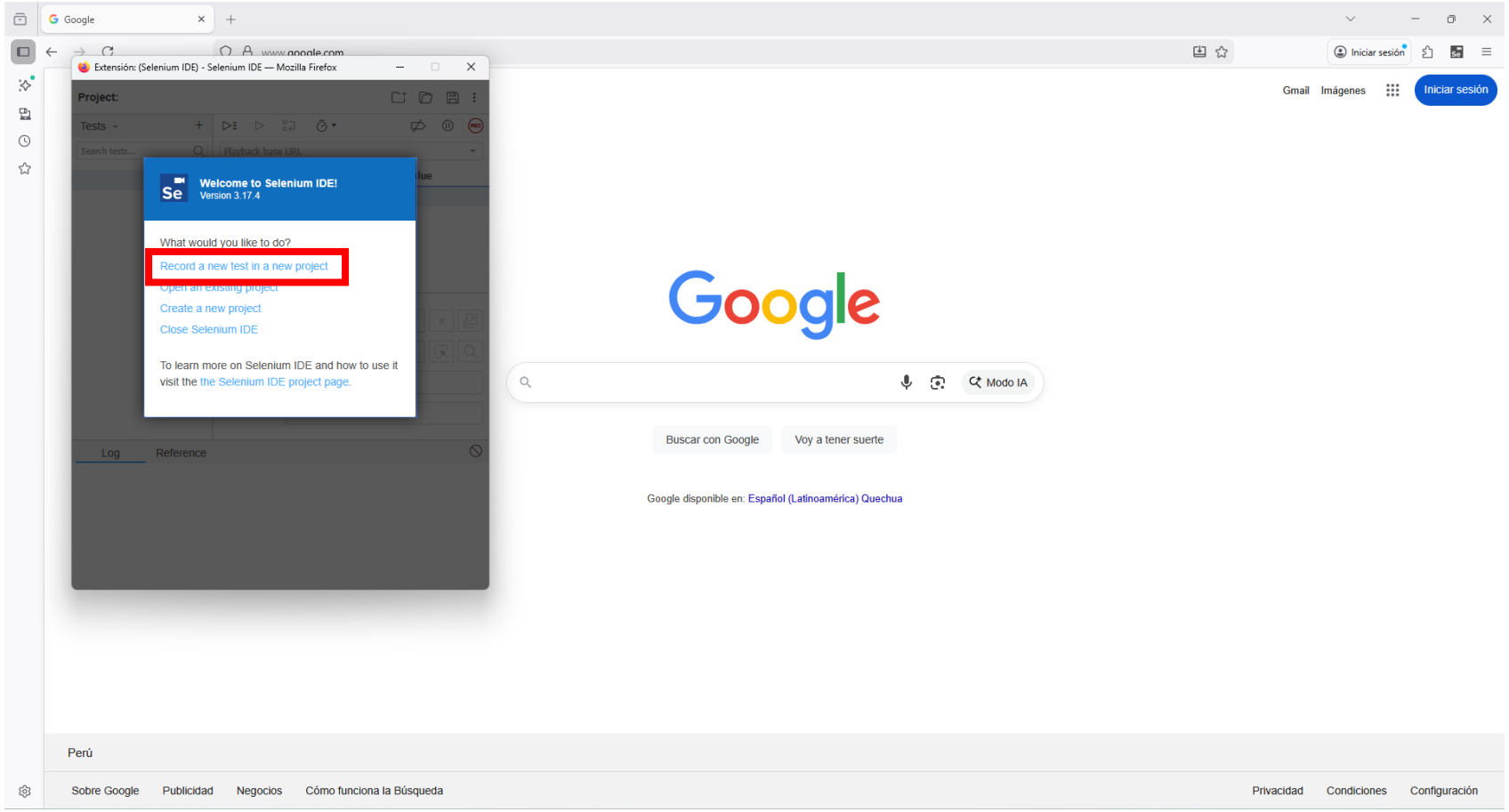
- Regresamos al Controlador
- Completamos el código

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    String sUsuario = request.getParameter("txtUsuario");
    String sPassword = request.getParameter("txtPassword");

    if (sUsuario.equals("admin") && sPassword.equals("12345")) {
        request.getRequestDispatcher("LoginOK.jsp").forward(request, response);
    }
    else {
        request.getRequestDispatcher("LoginError.jsp").forward(request, response);
    }
}
```

idat

Probamos nuestro proyecto

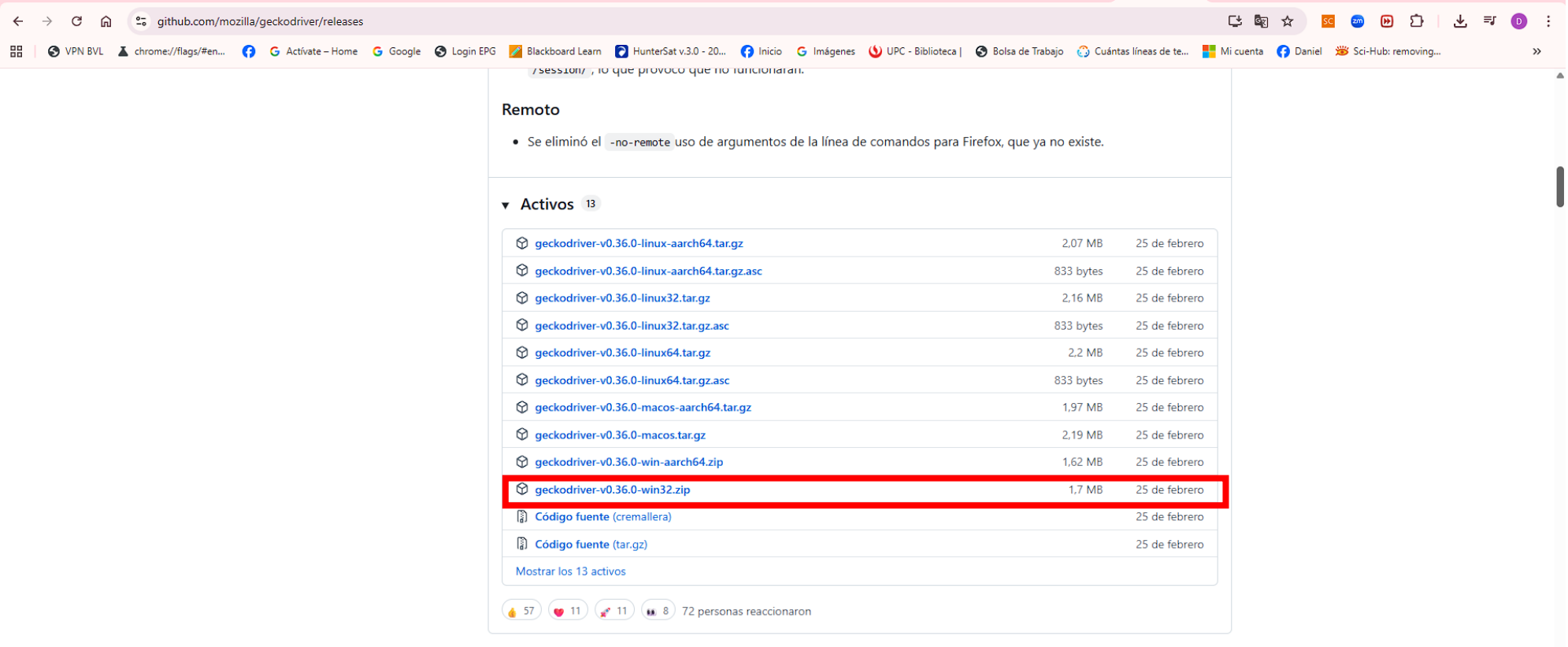
- http://localhost:8080/ejemploMiLogin/index.jsp



idat

Descargamos el Driver para Firefox

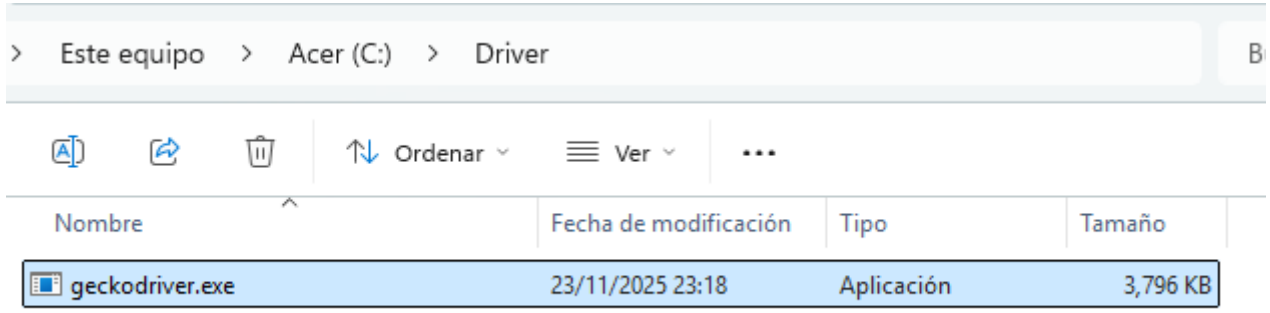
- https://github.com/mozilla/geckodriver/releases



idat

Copiamos el Driver para Firefox

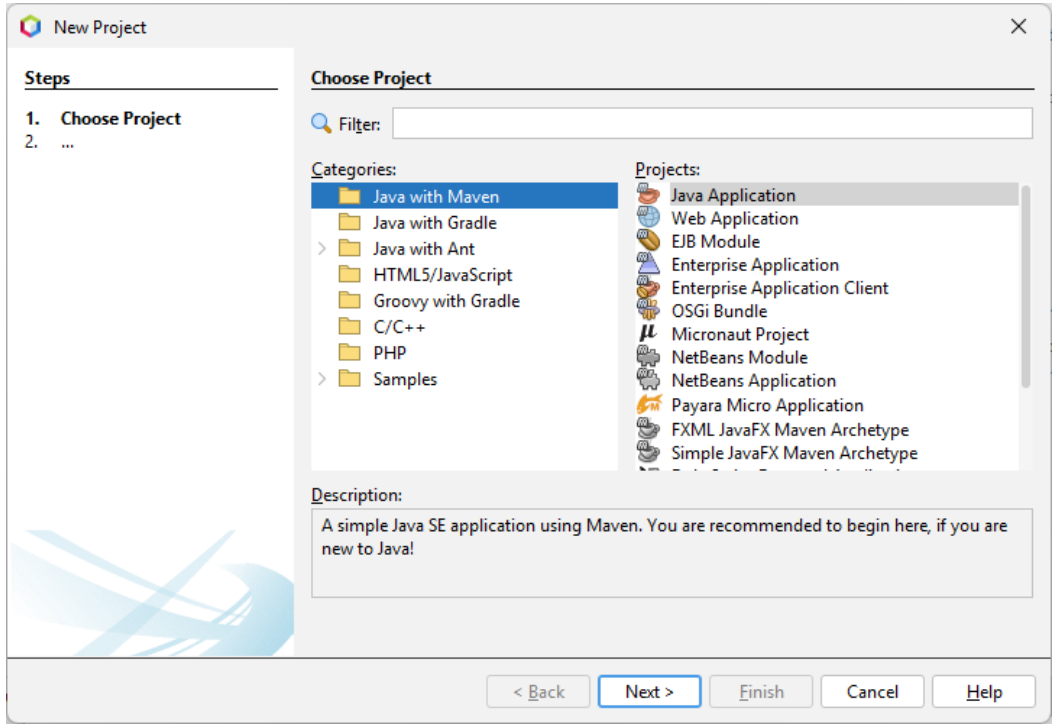
- Creamos una carpeta a la que podamos acceder y copiamos ahí el archivo descomprimido

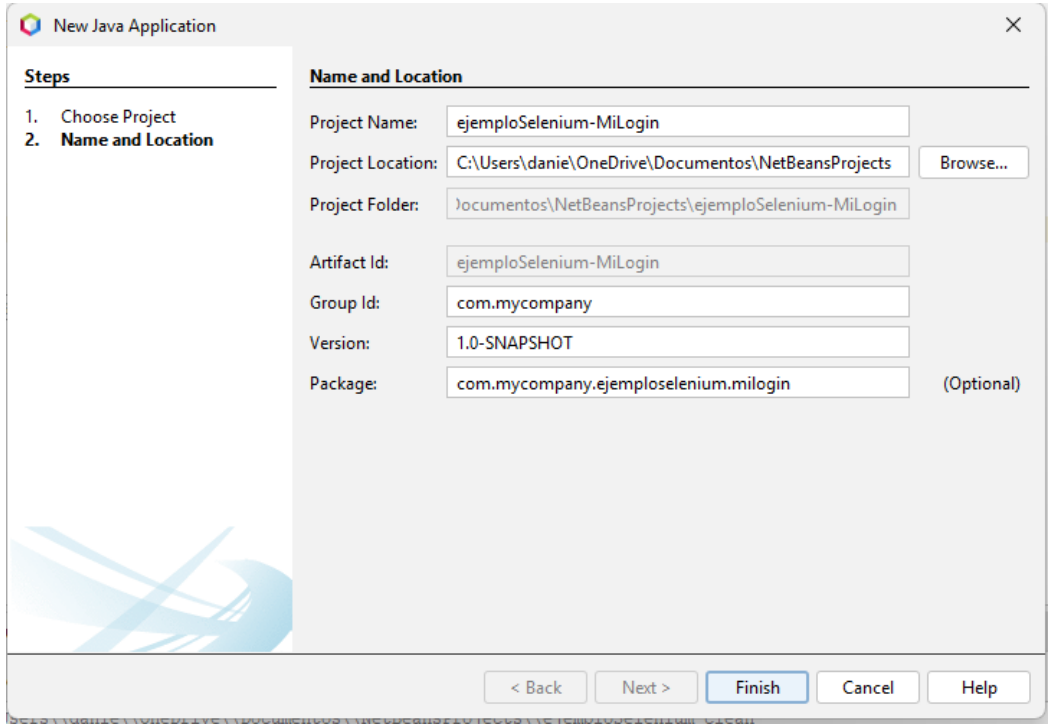


idat

Ejecutar Apache Netbeans

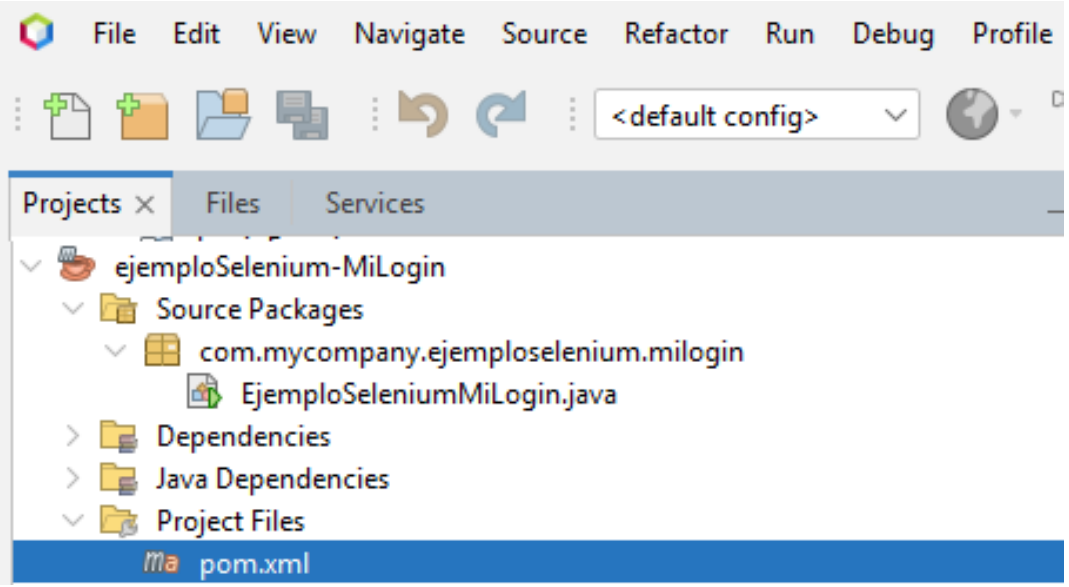
Creamos el Proyecto “ejemploSelenium-MiLogin”





idat

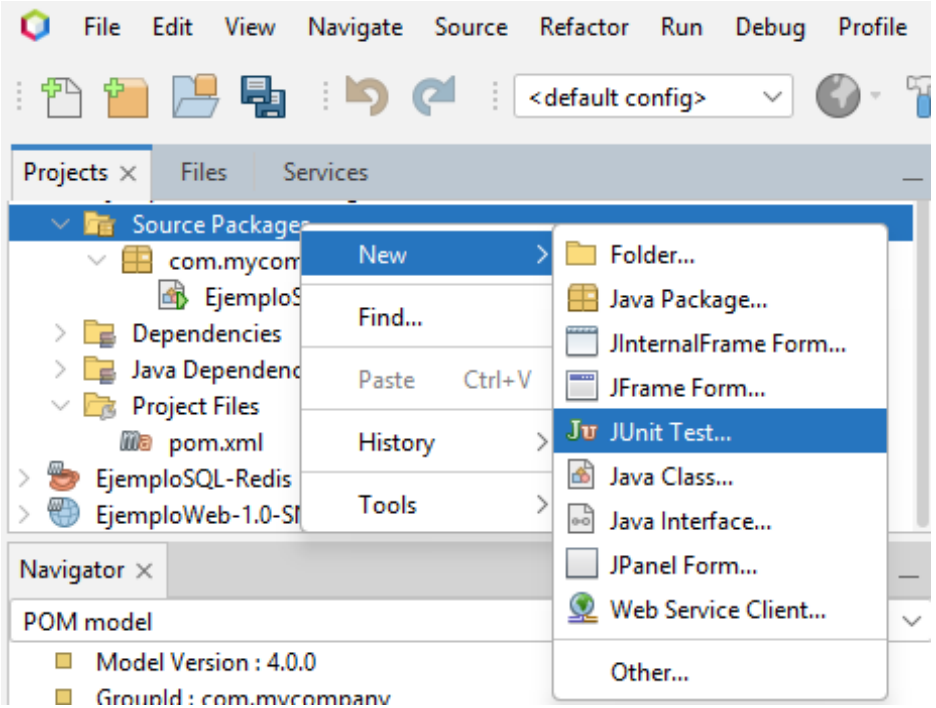
Modificar el pom.xml

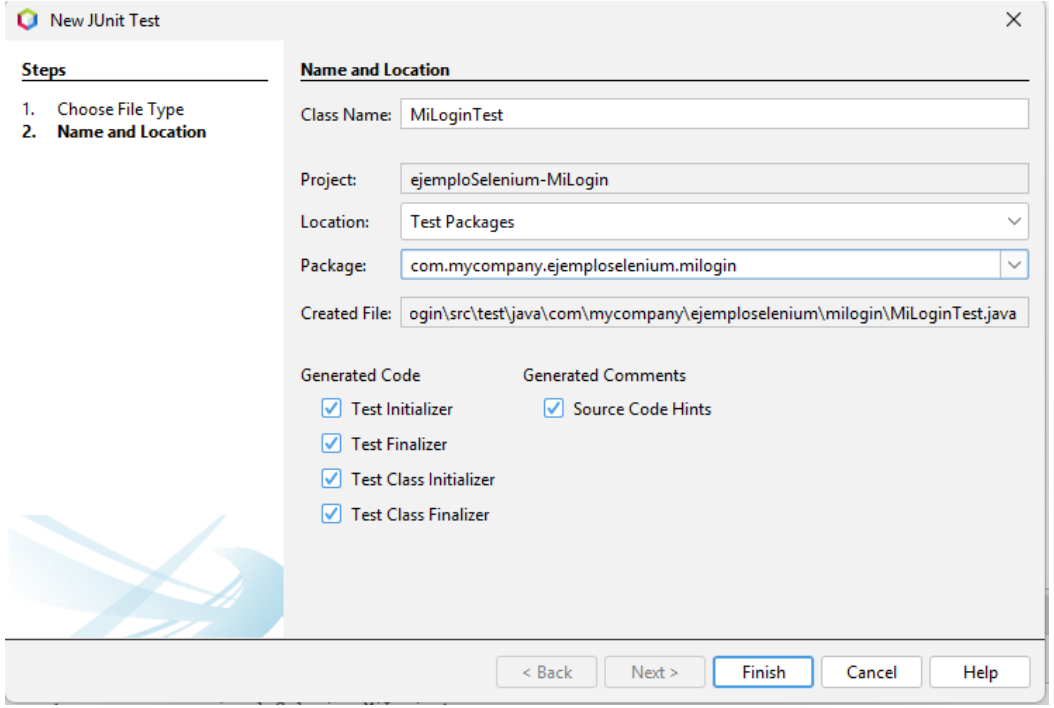



```xml
<dependencies>
    <!-- Selenium WebDriver -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.38.0</version>
    </dependency>

    <!-- Selenium Firefox Driver -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-firefox-driver</artifactId>
        <version>4.38.0</version>
    </dependency>

    <!-- JUnit -->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>5.10.3</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-params</artifactId>
        <version>5.10.3</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.10.3</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

idat

Creamos la Clase: MiLoginTest





idat

Codificamos la Clase: MiLoginTest

```
7   import org.junit.jupiter.api.Test;
8   import org.junit.jupiter.api.BeforeEach;
9   import org.junit.jupiter.api.AfterEach;
10  import static org.junit.jupiter.api.Assertions.*;
11
12  import org.openqa.selenium.By;
13  import org.openqa.selenium.WebDriver;
14  import org.openqa.selenium.WebElement;
15  import org.openqa.selenium.firefox.FirefoxDriver;
16  import org.openqa.selenium.Dimension;
17  import org.openqa.selenium.JavascriptExecutor;
18  import org.openqa.selenium.support.ui.ExpectedConditions;
19  import org.openqa.selenium.support.ui.WebDriverWait;
20
21  import java.time.Duration;
22  import java.util.HashMap;
23  import java.util.Map;
```

idat

Codificamos la Clase: MiLoginTest

```
25  public class MiLoginTest {
26
27      public MiLoginTest() {
28      }
29
30      private WebDriver driver;
        private Map<String, Object> vars;
        JavascriptExecutor js;
33      private WebDriverWait wait;
34
35      @BeforeEach
36      public void setUp() {
37          // Configurar Firefox Driver
38          driver = new FirefoxDriver();
39          js = (JavascriptExecutor) driver;
            vars = new HashMap<String, Object>();
41
42          // Configurar wait explícito de 10 segundos
43          wait = new WebDriverWait(driver, Duration.ofSeconds(10));
44
45          // Configurar tiempo de espera implícito
46          driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
47      }
48
49      @AfterEach
50      public void tearDown() {
51          // Cerrar el navegador después de cada test
52          if (driver != null) {
53              driver.quit();
54          }
55      }
```

idat

Codificamos la Clase: MiLoginTest

```java
57      @Test
        public void testLogin() {
59          try {
60              // Paso 1: Navegar a la página de login
61              System.out.println("Navegando a la página de login...");
62              driver.get("http://localhost:8080/ejemploMiLogin/index.jsp");
63
64              // Verificar que estamos en la página correcta
65              assertTrue(driver.getCurrentUrl().contains("index.jsp"),
66                         "Debe estar en la página de login");
67
68              // Paso 2: Configurar tamaño de ventana
69              driver.manage().window().setSize(new Dimension(714, 852));
70
71              // Paso 3: Primer intento de login con credenciales incorrectas
72              System.out.println("Primer intento con credenciales incorrectas...");
73
74              WebElement txtUsuario = wait.until(
75                  ExpectedConditions.elementToBeClickable(By.id("txtUsuario"))
76              );
77              txtUsuario.click();
78              txtUsuario.clear();
79              txtUsuario.sendKeys("usu");
80
81              WebElement txtPassword = driver.findElement(By.id("txtPassword"));
82              txtPassword.clear();
83              txtPassword.sendKeys("123");
84
85              WebElement btnLogin = driver.findElement(By.id("btnEnviar"));
86              btnLogin.click();
87
88              // Esperar a que se procese el login
89              Thread.sleep(2000);
90
91              // Paso 4: Volver a la página de login
92              System.out.println("Volviendo a la página de login...");
93              WebElement volverLink = wait.until(
94                  ExpectedConditions.elementToBeClickable(By.linkText("Volver..."))
95              );
96              volverLink.click();
```

```java
98              // Verificar que volvimos al login
99              wait.until(ExpectedConditions.presenceOfElementLocated(By.id("txtUsuario")));
100
101             // Paso 5: Segundo intento con credenciales correctas
102             System.out.println("Segundo intento con credenciales correctas...");
103
104             WebElement txtUsuario2 = wait.until(
105                 ExpectedConditions.elementToBeClickable(By.id("txtUsuario"))
106             );
107             txtUsuario2.click();
108             txtUsuario2.clear();
109             txtUsuario2.sendKeys("admin");
110
111             WebElement txtPassword2 = driver.findElement(By.id("txtPassword"));
112             txtPassword2.clear();
113             txtPassword2.sendKeys("12345");
114
115             WebElement btnLogin2 = driver.findElement(By.id("btnEnviar"));
116             btnLogin2.click();
117
118             // Esperar a que se procese el login exitoso
119             Thread.sleep(2000);
120
121             // Paso 6: Verificar login exitoso y volver
122             System.out.println("Verificando login exitoso...");
123             WebElement volverLink2 = wait.until(
124                 ExpectedConditions.elementToBeClickable(By.linkText("Volver..."))
125             );
126             volverLink2.click();
127
128             // Verificación final - debemos estar de vuelta en el login
129             WebElement usuarioField = wait.until(
130                 ExpectedConditions.presenceOfElementLocated(By.id("txtUsuario"))
131             );
132             assertTrue(usuarioField.isDisplayed(),
133                        "Debe estar de vuelta en la página de login");
134
135             System.out.println("Prueba completada exitosamente!");
136
            } catch (Exception e) {
138             fail("La prueba falló con error: " + e.getMessage());
                e.printStackTrace();
140         }
141     }
```

idat

Codificamos la Clase: MiLoginTest

```
143         @Test
            public void testCamposLoginPresentes() {
145             try {
146                 // Navegar a la página
147                 driver.get("http://localhost:8080/ejemploMiLogin/index.jsp");
148
149                 // Verificar que todos los elementos del login estén presentes
150                 WebElement usuarioField = wait.until(
151                     ExpectedConditions.presenceOfElementLocated(By.id("txtUsuario"))
152                 );
153                 assertTrue(usuarioField.isDisplayed(), "Campo usuario debe estar presente");
154
155                 WebElement passwordField = driver.findElement(By.id("txtPassword"));
156                 assertTrue(passwordField.isDisplayed(), "Campo password debe estar presente");
157
158                 WebElement loginButton = driver.findElement(By.id("btnEnviar"));
159                 assertTrue(loginButton.isDisplayed(), "Botón login debe estar presente");
160
161                 System.out.println("Todos los elementos del login están presentes correctamente");
162
163             } catch (Exception e) {
164                 fail("Test de elementos falló: " + e.getMessage());
165             }
166         }
167     }
```

idat

¿Dudas?

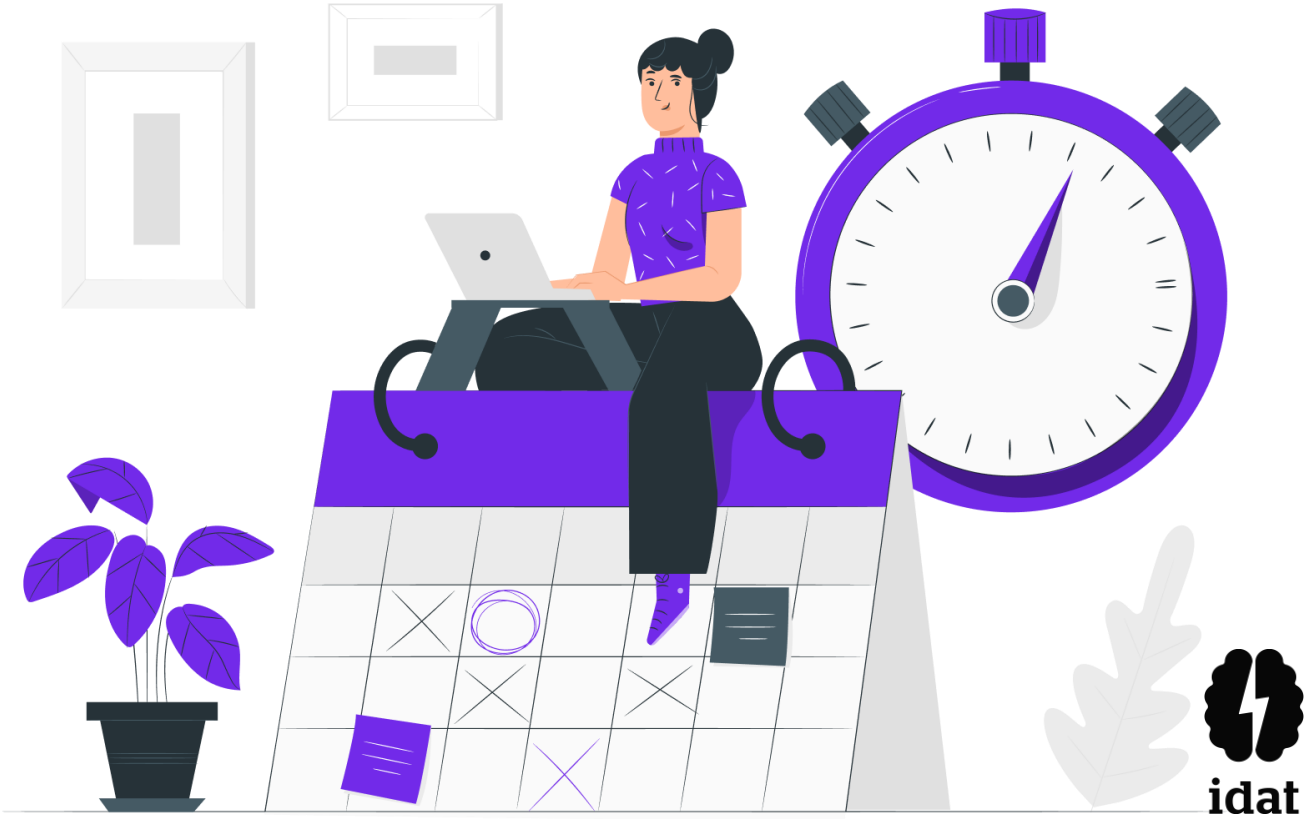
¿Miedos?

¿Temores?

idat

Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat

idat

GRACIAS